



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
**«ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
ДВАЖДЫ ГЕРОЯ СОВЕТСКОГО СОЮЗА,
ЛЕТЧИКА-КОСМОНАВТА А.А. ЛЕОНОВА» -**
ФИЛИАЛ ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО ОБРАЗОВАТЕЛЬНОГО
УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ

**«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ ГЕОДЕЗИИ И КАРТОГРАФИИ»
ИНСТИТУТ ИНФОКОММУНИКАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ**

КАФЕДРА МАТЕМАТИКИ И ЕСТЕСТВЕННОНАУЧНЫХ ДИСЦИПЛИН

Допустить к защите

« ____ » _____ 2026 г.

Заведующий кафедрой _____ И. В. Бугай

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Тема: разработка программы управления ходовой частью наземного

четырёхколёсного робота на базе микроконтроллеров stm32

Направление подготовки 01.03.02 Прикладная математика и информатика

Выполнил(а): студент(ка)

Михно Павел Юрьевич

Группа: ПМИ-22, 4 курс

Научный руководитель: к.т.н, доцент, И.В. Бугай

Королёв

2026

Содержание

Введение.....	4
1. Описание мобильной платформы.....	7
2. Математическая модель и синтез управления платформой.....	13
2.1. Кинематическая модель мобильной платформы.....	14
2.2. Постановка задачи управления.....	17
2.3. Синтез закона управления.....	20
2.4. Выводы.....	25
3. Разработка программы.....	27
3.1. Назначение программы.....	27
3.2. Функциональные требования к программе.....	28
3.3. Архитектура программы.....	30
3.3.1. Подсистема организации управления и представления состояния платформы.....	33
3.3.2. Подсистема реализации закона управления.....	34
3.3.3. Подсистема аппаратной абстракции.....	35
3.3.4. Выводы.....	36
3.4. Программная реализация цикла управления.....	37
3.4.1. Общий шаг управления.....	37
3.4.2. Формирование текущего скоростного состояния платформы.....	39
3.4.3. Вычисление управляющего воздействия.....	42
3.4.4. Передача управляющего воздействия и обработка недопустимых состояний.....	45
3.4.5. Выводы.....	47
3.5. Тестирование.....	48
3.5.1. Модульное тестирование.....	48
3.5.2. Тестирование в среде моделирования.....	50

3.5.2.1. Интеграция рабочего контура управления со средой моделирования.....	51
3.5.2.2. Методика экспериментальной оценки.....	53
3.5.2.3. Результаты испытаний.....	57
3.5.2.3.1. Прямолинейное движение.....	58
3.5.2.3.2. Поворот на месте.....	60
3.5.2.3.3. Движение по квадратной траектории.....	62
3.5.2.4. Выводы по испытаниям в среде моделирования.....	66
3.6. Выводы.....	67
4. Встраиваемая реализация и аппаратно-программная проверка программы управления на платформе STM32.....	70
4.1. Организация аппаратно-программного контура проверки.....	70
4.2. Реализация прошивки и обмена данными.....	72
4.3. Методика проверки.....	75
4.4. Результаты аппаратно-программной проверки.....	76
4.5. Границы применимости результатов.....	83
4.6. Выводы.....	83
Заключение.....	85
Список источников.....	87

Введение

Мобильные робототехнические платформы широко применяются в задачах транспортировки, обследования пространства и выполнения автономных операций. Возможность их практического использования в значительной степени определяется качеством управления ходовой частью, обеспечивающей поступательное движение и выполнение манёвров [8, 11]. Для колёсного мобильного робота с электрическим приводом такая задача связана с формированием управляющих воздействий на приводы по заданным параметрам движения и использованием обратной связи для компенсации отклонений фактического режима от требуемого [1, 3, 10].

Разработка полноценной мобильной робототехнической платформы представляет собой комплексную задачу, включающую проектирование механической части, выбор приводов и датчиков, создание встраиваемого программного обеспечения, реализацию алгоритмов управления, навигации и взаимодействия с внешней средой. Поэтому целесообразным является поэтапный подход, при котором на начальном этапе разрабатывается и проверяется базовый контур управления ходовой частью, а затем на его основе создаётся физический прототип робота и реализуются функции более высокого уровня.

В настоящей работе рассматривается первый этап разработки системы управления четырёхколёсным мобильным роботом с дифференциальной схемой привода. На данном этапе разрабатывается программный контур скоростного управления, формируется его архитектура, выбирается предварительная математическая модель и проверяется выполнение вычислительной логики в среде моделирования и на микроконтроллерной платформе STM32.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью создания и проверки программной и архитектурной основы системы управления мобильным роботом до создания его физического прототипа.

Объектом исследования является ходовая часть четырёхколёсной мобильной робототехнической платформы с дифференциальной схемой привода.

Предметом исследования является программа управления ходовой частью мобильной платформы, формирующая воздействия на приводы по заданным линейной и угловой скоростям платформы с использованием математической модели и обратной связи по данным энкодеров ведущих колёс.

Целью выпускной квалификационной работы является разработка и аппаратно-программная проверка базового программного контура скоростного управления четырёхколёсной мобильной платформой на базе STM32, предназначенного для последующего уточнения и применения при создании физического прототипа робота.

В работе рассматривается задача скоростного управления на основе предварительной модели мобильной платформы. Управление положением платформы в глобальной системе координат, экспериментальная идентификация параметров физического объекта и проверка системы на реальной ходовой части не входят в предмет настоящей работы.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Рассмотреть ходовую часть четырёхколёсной мобильной платформы с дифференциальной схемой привода как объект управления.
2. Построить предварительную математическую модель платформы с дифференциальной схемой привода.
3. Сформулировать задачу скоростного управления и разработать закон управления движением платформы.
4. Определить функциональные требования и разработать архитектуру рабочего контура управления.
5. Реализовать дискретный программный цикл управления с обратной связью по данным энкодеров ведущих колёс.

6. Выполнить модульную проверку компонентов рабочего контура и испытания его совместной работы в среде моделирования Webots.
7. Реализовать выполнение вычислительной части управления на платформе STM32 и провести её аппаратно-программную проверку.

Практическая значимость работы состоит в создании программной основы рабочего контура скоростного управления, пригодной для последующей интеграции с физической мобильной платформой. После создания физического прототипа потребуется провести экспериментальную идентификацию параметров объекта управления, уточнить математическую модель и повторно настроить регуляторы с учётом характеристик реальных приводов, датчиков и механической конструкции.

1. Описание мобильной платформы

В качестве объекта управления в работе рассматривается мобильная робототехническая платформа, предназначенная для перемещения в плоскости с заданными параметрами поступательного и вращательного движения. С функциональной точки зрения данная платформа представляет собой механическую систему, способную изменять своё положение и ориентацию под действием управляющих воздействий, формируемых программной системой управления ходовой частью. Следовательно, в рамках настоящей работы робот рассматривается как объект автоматического управления, для которого требуется обеспечить воспроизведение заданного режима движения [1, 11].

Конструктивно рассматриваемая платформа выполнена по четырёхколёсной схеме. В состав ходовой части входят два ведущих колеса, расположенных по бокам корпуса, и два опорных роликовых колеса, размещённых в передней и задней частях платформы. Ведущие колёса являются активными исполнительными элементами и непосредственно участвуют в формировании движения робота. Опорные роликовые колёса собственного привода не имеют и выполняют вспомогательную функцию, связанную с обеспечением устойчивости конструкции, поддержанием корпуса относительно опорной поверхности и перераспределением нагрузки [2, 11].

С точки зрения кинематики рассматриваемая платформа относится к классу мобильных роботов с дифференциальной схемой привода. Особенность данной схемы состоит в том, что движение платформы в плоскости определяется независимым управлением двумя ведущими колёсами. Иначе говоря, общее движение робота формируется как результат совместной работы двух приводов, а изменение режима движения достигается изменением скоростей вращения левого и правого колёс. Такая схема широко применяется в мобильной робототехнике благодаря конструктивной простоте, удобству математического описания и возможности реализации основных режимов движения без усложнения механической структуры объекта [2, 5, 8].

Физический смысл движения при дифференциальной схеме привода определяется соотношением между скоростями ведущих колёс. Если скорости левого и правого колеса совпадают по величине и направлению, платформа движется прямолинейно. Если скорости различаются, движение становится криволинейным, поскольку возникает вращение корпуса относительно мгновенного центра поворота. Если же скорости колёс равны по модулю, но противоположны по знаку, реализуется разворот на месте без поступательного перемещения платформы. Таким образом, изменение характера движения достигается без изменения геометрии ходовой части, исключительно за счёт изменения относительных скоростей двух исполнительных каналов.

Выбор именно дифференциальной схемы привода в качестве основы рассматриваемой платформы обусловлен её конструктивной простотой, удобством инженерного анализа и естественным соответствием задаче разработки программной системы управления ходовой частью. Такая платформа допускает переход от команд верхнего уровня, задаваемых в виде линейной и угловой скоростей, к управляющим воздействиям на конкретные исполнительные механизмы. По этой причине она является удобным объектом для построения математической модели и последующего синтеза закона управления [2, 5, 8].

С позиций теории автоматического управления рассматриваемая платформа представляет собой управляемую механическую систему, состояние которой изменяется под действием входных воздействий. Входами объекта управления являются управляющие сигналы, подаваемые на приводы ведущих колёс. В зависимости от уровня абстракции эти воздействия могут интерпретироваться как сигналы на двигатели, как требуемые угловые скорости вращения колёс либо как обобщённые линейная и угловая скорости платформы. Выходами системы являются измеряемые или оцениваемые параметры движения, к которым относятся угловые скорости ведущих колёс, линейная и

угловая скорости платформы, а также её положение и ориентация в плоскости [1, 10].

В рамках настоящей работы корпус платформы рассматривается как абсолютно жёсткое тело, движение которого ограничено плоскостью. Такое допущение позволяет не учитывать вертикальные перемещения, наклоны корпуса, деформации конструкции и другие пространственные эффекты, несущественные для поставленной задачи. При этом положение платформы может быть описано координатами характерной точки корпуса в плоскости и углом ориентации относительно выбранного направления отсчёта. Подобная постановка соответствует принятому далее кинематическому описанию объекта управления [5, 8].

Принципиально важно, что в рассматриваемой платформе активными исполнительными звеньями ходовой части являются только два ведущих колеса. Опорные роликовые элементы не участвуют в создании тягового усилия и не формируют собственного управляющего воздействия на движение робота. Их влияние на поведение платформы учитывается косвенно — через предположение о сохранении устойчивого контакта с опорной поверхностью. Это позволяет в дальнейшем сосредоточить математическое описание на связи между движением ведущих колёс и движением платформы как единого объекта.

С инженерной точки зрения поведение рассматриваемой платформы не может быть полностью описано её номинальной математической моделью. На движение платформы оказывают влияние погрешности измерений, неидеальность приводов, отклонение параметров реального объекта от расчётных значений, а также внешние возмущения. Поэтому для компенсации отклонений фактического движения от заданного в системе управления используется корректирующее воздействие, формируемое по сигналам обратной связи. Это обстоятельство имеет принципиальное значение для дальнейшего построения системы управления, в которой модельное описание используется

для расчёта номинального управляющего воздействия, а регулятор — для компенсации возникающих отклонений [1, 3].

Для дальнейшего построения алгоритма управления существенно, что движение платформы может быть описано через обобщённые параметры — линейную и угловую скорости. Такое представление удобно как с точки зрения математического моделирования, так и с точки зрения программной реализации, поскольку позволяет верхнему уровню системы управления оперировать режимом движения платформы как единого объекта, а не параметрами каждого колеса по отдельности. Именно это представление используется в последующих разделах работы при постановке задачи управления и разработке программной реализации вычислительного контура.

В рамках данной работы для описания движения платформы используется кинематическая модель. Такой подход означает, что основное внимание уделяется связи между параметрами движения ведущих колёс и геометрией движения платформы без явного учёта сил, моментов и инерционных эффектов. Выбор именно кинематического описания обусловлен характером решаемой задачи, связанной с разработкой программной системы управления движением на уровне скоростных команд. Использование полной динамической модели в данной постановке не является необходимым, поскольку привело бы к существенному усложнению математического описания и алгоритмической реализации без принципиального выигрыша для рассматриваемого класса задач [5, 8].

Следует, однако, отметить, что кинематическое описание опирается на ряд упрощающих предположений. Предполагается отсутствие существенного проскальзывания ведущих колёс относительно опорной поверхности, постоянство геометрических параметров платформы, движение по ровной поверхности и пренебрежение рядом эффектов динамической природы, способных повлиять на фактическое поведение системы. Такие допущения допустимы для синтеза базового закона управления, однако их приближённый

характер означает, что математическая модель не может полностью совпадать с реальным объектом во всех режимах работы. Поэтому при практической реализации системы управления требуется использование обратной связи, компенсирующей отклонения, не учитываемые номинальной моделью [1, 8].

Общий вид рассматриваемой мобильной платформы и расположение основных элементов её ходовой части представлены на рисунке 1.

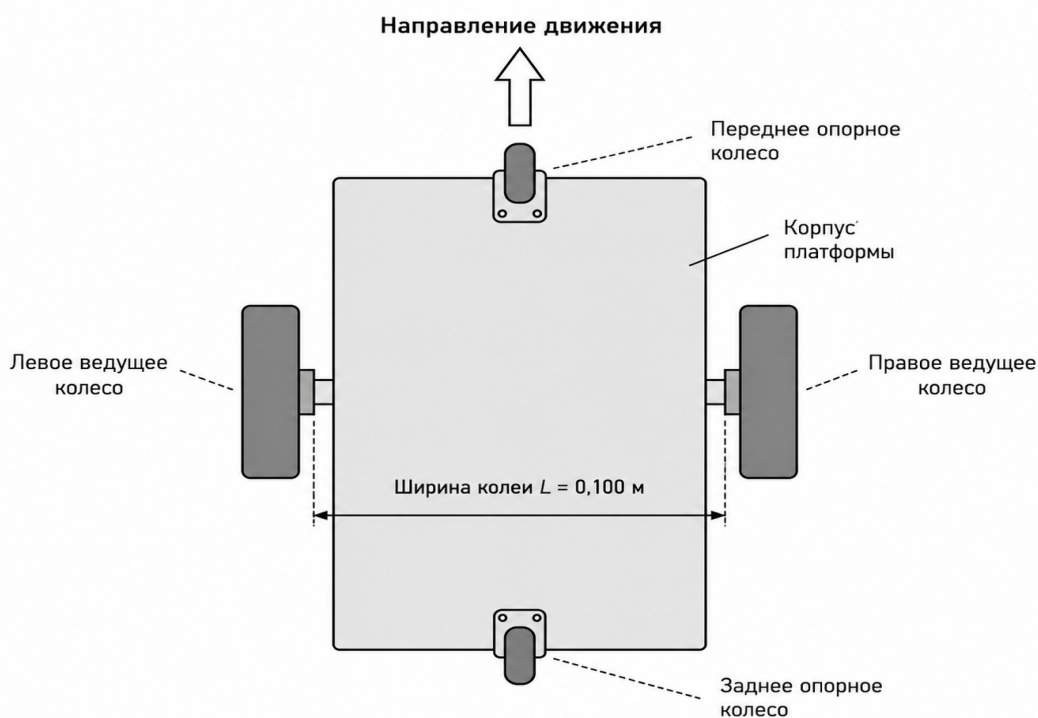


Рисунок 1 — Изображение мобильной платформы с дифференциальной схемой привода

Представленная конструктивная схема показывает, что активное формирование движения платформы осуществляется двумя ведущими колёсами, тогда как остальные элементы ходовой части выполняют вспомогательные функции. Такое устройство платформы определяет выбор её кинематического описания и служит основанием для дальнейшего построения математической модели объекта управления.

Таким образом, рассматриваемая мобильная платформа представляет собой типичный, но в то же время достаточно содержательный объект управления для задач разработки программного обеспечения ходовой части. Она

допускает сравнительно простое математическое описание, естественно согласуется с представлением движения в виде линейной и угловой скоростей и позволяет реализовать программную систему управления, в которой математическая модель используется для формирования номинального режима движения, а обратная связь — для компенсации отклонений от него.

2. Математическая модель и синтез управления платформой

Разработка программы управления ходовой частью мобильного робота требует предварительного математического описания объекта управления и формализации требований к его движению. Без такого описания невозможно обоснованно определить состав входных и выходных переменных системы, установить взаимосвязь между задаваемыми параметрами движения и фактически реализуемыми воздействиями на приводы, а также построить закон управления, обеспечивающий воспроизведение требуемого режима движения [1].

В рамках настоящей работы в качестве объекта управления рассматривается мобильная платформа с дифференциальной схемой привода, конструктивные особенности которой были приведены в предыдущей главе. В качестве математического описания объекта управления используется кинематическая модель мобильной платформы. Выбор именно кинематической модели обусловлен тем, что решаемая задача связана с формированием режима движения платформы по заданным линейной и угловой скоростям, а не с детальным исследованием силовых процессов в приводах и механической системе. При принятых допущениях о движении по плоской поверхности, отсутствии существенного бокового проскальзывания и работе в режимах, для которых определяющими являются кинематические ограничения, такая модель позволяет установить однозначную связь между параметрами движения платформы и угловыми скоростями ведущих колёс при существенно меньшей сложности по сравнению с полной динамической моделью [2, 5, 8].

Следует отметить, что использование только номинальной математической модели не позволяет в полном объёме учесть реальные условия функционирования системы. На движение мобильной платформы оказывают влияние погрешности задания параметров, неточность коэффициентов модели, особенности приводов и внешние возмущающие воздействия. По этой причине в работе применяется комбинированный подход к построению управления:

номинальные управляющие значения определяются на основе математической модели, а отклонения фактического движения от требуемого компенсируются с помощью обратной связи. Такая структура позволяет совместить простоту модельного расчёта с повышением точности управления при наличии отклонений фактического поведения объекта от номинальной модели [1, 3].

2.1. Кинематическая модель мобильной платформы

При построении математической модели принимаются следующие допущения: платформа рассматривается как абсолютно твёрдое тело, движение происходит по плоской поверхности, боковое проскальзывание отсутствует, а радиусы ведущих колёс и геометрические параметры платформы считаются постоянными. Указанные допущения позволяют использовать кинематическую модель, на основе которой определяются требуемые скорости ведущих колёс, используемые далее при формировании номинальной составляющей управления и её последующей коррекции по обратной связи [5, 8].

Положение платформы в неподвижной системе координат определяется координатами характерной точки платформы и углом её ориентации. В качестве вектора состояния примем

$$q(t) = \begin{pmatrix} x(t) \\ y(t) \\ \theta(t) \end{pmatrix}$$

где $x(t)$ и $y(t)$ — координаты характерной точки платформы в плоскости движения, а $\theta(t)$ — угол ориентации продольной оси платформы относительно неподвижной системы координат [5, 8].

Для описания движения рассмотрим две системы координат: неподвижную глобальную систему координат, связанную с рабочим пространством, и локальную систему координат, связанную с платформой. В локальной системе координат поступательное движение платформы направлено вдоль её продольной оси, а вращательное движение характеризуется угловой скоростью вокруг вертикальной оси. Если линейная скорость платформы вдоль продольной

оси равна $v(t)$, а угловая скорость — $\omega(t)$, то переход к неподвижной системе координат даёт следующие кинематические уравнения [5, 8]:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \omega$$

Первое уравнение показывает, что изменение координаты x определяется проекцией линейной скорости платформы на ось абсцисс неподвижной системы координат. Аналогично, второе уравнение задаёт изменение координаты y как проекцию линейной скорости на ось ординат. Третье уравнение отражает изменение ориентации платформы во времени и непосредственно задаётся её угловой скоростью.

Таким образом, движение платформы полностью определяется двумя величинами: линейной скоростью $v(t)$ и угловой скоростью $\omega(t)$. Однако в рассматриваемой системе эти величины не задаются непосредственно исполнительным механизмом. Управляющее воздействие формируется через приводы левого и правого ведущих колёс, поэтому необходимо установить связь между скоростями платформы как целого и скоростями отдельных колёс [2, 5, 8].

Обозначим через $v_L(t)$ и $v_R(t)$ линейные скорости точек контакта левого и правого ведущих колёс с опорной поверхностью, а через L — расстояние между ведущими колёсами. При прямолинейном движении платформы скорости обоих колёс совпадают, и платформа движется без изменения ориентации. При различии скоростей колёс возникает вращение платформы. В рамках кинематики дифференциального привода линейная и угловая скорости платформы определяются выражениями [2, 5, 8]:

$$v = \frac{v_R + v_L}{2}$$

$$\omega = \frac{v_R - v_L}{L}$$

Первое соотношение означает, что линейная скорость платформы равна средней линейной скорости ведущих колёс. Это следует из того, что при отсутствии бокового проскальзывания общее поступательное движение формируется совместным действием обоих приводов. Второе соотношение показывает, что угловая скорость платформы пропорциональна разности скоростей правого и левого колёс: чем больше эта разность, тем интенсивнее поворот платформы.

Для последующего решения задачи управления требуется также обратное преобразование, то есть определение скоростей колёс по заданным линейной и угловой скоростям платформы. Решая приведённую систему относительно $v_L(t)$ и $v_R(t)$, получаем [2, 5, 8]:

$$v_L = v - \frac{L \omega}{2}$$

$$v_R = v + \frac{L \omega}{2}$$

Эти выражения имеют очевидный физический смысл. При положительной угловой скорости $\omega(t)$ платформа должна поворачиваться в одну сторону, что достигается уменьшением скорости одного колеса и увеличением скорости другого. При $\omega(t)=0$ обе скорости становятся равными, и движение происходит прямолинейно. При $v(t)=0$ и ненулевой $\omega(t)$ формируется режим разворота на месте, когда колёса вращаются в противоположных направлениях [2, 8].

Поскольку в программной реализации и при работе с приводами удобно использовать угловые скорости вращения колёс, линейные скорости $v_L(t)$ и $v_R(t)$ дополнительно выражаются через угловые скорости $\omega_L(t)$ и $\omega_R(t)$. Пусть r — радиус ведущего колеса. Тогда [2, 8]:

$$\omega_L(t) = \frac{v_L(t)}{r},$$

$$\omega_R(t) = \frac{v_R(t)}{r}.$$

Подставляя сюда найденные ранее выражения для линейных скоростей колёс, получаем

$$\omega_L = \frac{1}{r} \left(v - \frac{L \omega}{2} \right)$$

$$\omega_R = \frac{1}{r} \left(v + \frac{L \omega}{2} \right)$$

Полученные зависимости являются ключевыми для дальнейшего построения системы управления. Они устанавливают однозначную связь между требуемыми параметрами движения платформы и угловыми скоростями вращения её ведущих колёс. Иными словами, кинематическая модель позволяет перейти от описания движения платформы как единого объекта к формированию опорных значений для каждого привода, которые в дальнейшем используются при синтезе закона управления.

2.2. Постановка задачи управления

После построения кинематической модели объекта управления необходимо формализовать задачу управления, то есть определить задающие и регулируемые величины, а также требуемый результат функционирования системы [1].

В рамках рассматриваемой системы на вход программы управления поступают заданные значения линейной и угловой скоростей платформы

$$u(t) = \begin{pmatrix} v_{\text{зад}}(t) \\ \omega_{\text{зад}}(t) \end{pmatrix}$$

Указанные сигналы определяют требуемый режим движения мобильной платформы в плоскости. Заданная линейная скорость характеризует интенсивность поступательного движения, а заданная угловая скорость — интенсивность изменения ориентации платформы. Совокупность этих величин полностью определяет желаемое кинематическое поведение объекта управления в каждый момент времени [5, 8].

Однако непосредственное управление исполнительными механизмами по величинам $v(t)$ и $\omega(t)$ невозможно, поскольку приводы воздействуют не на платформу как на единый объект, а на её отдельные ведущие колёса. Поэтому первой частью задачи управления является преобразование заданных параметров движения платформы в параметры движения ведущих колёс. На основе кинематической модели для дифференциальной платформы требуемые угловые скорости левого и правого ведущих колёс определяются выражениями [2, 5, 8]:

$$\omega_{L,зад}(t) = \frac{1}{r} \left(v_{зад}(t) - \frac{L \omega_{зад}(t)}{2} \right)$$

$$\omega_{R,зад}(t) = \frac{1}{r} \left(v_{зад}(t) + \frac{L \omega_{зад}(t)}{2} \right)$$

Указанное преобразование необходимо для формирования номинального режима работы приводов, соответствующего заданному движению платформы. Однако в текущей реализации системы управления обратная связь организована не по угловым скоростям отдельных колёс, а по линейной и угловой скоростям платформы как единого объекта. Такой подход соответствует общей структуре управления движением, в которой объектом регулирования выступает кинематическое состояние платформы в целом, а не каждый привод изолированно [1, 3].

Фактические линейная и угловая скорости платформы определяются на основе данных энкодеров ведущих колёс. При этом фактическая линейная скорость платформы обозначается через $v(t)$, а фактическая угловая скорость — через $\omega(t)$. Ошибки регулирования определяются как отклонения фактических скоростей платформы от их заданных значений [1, 3]:

$$e_v(t) = v_{зад}(t) - v(t)$$

$$e_\omega(t) = \omega_{зад}(t) - \omega(t)$$

Эти выражения отражают степень отклонения реального движения платформы от требуемого режима. Если ошибки равны нулю, фактическое

движение платформы полностью соответствует задаваемым параметрам. При ненулевых значениях ошибок система управления должна формировать корректирующее воздействие, направленное на уменьшение рассогласования.

Следовательно, задача управления в рассматриваемой работе формулируется следующим образом: требуется построить такой закон управления, при котором фактические линейная и угловая скорости мобильной платформы $v(t)$ и $\omega(t)$ стремятся к соответствующим заданным значениям $v_{зад}(t)$ и $\omega_{зад}(t)$. Формально цель управления может быть записана в виде [1]:

$$\lim_t v(t) = v_{зад}(t)$$

$$\lim_t \omega(t) = \omega_{зад}(t)$$

Поскольку линейная и угловая скорости платформы однозначно связаны с угловыми скоростями ведущих колёс, выполнение указанных условий означает воспроизведение требуемого режима движения мобильной платформы в целом. Тем самым кинематическая модель используется для вычисления номинальных параметров движения колёс, а обратная связь — для обеспечения соответствия фактического движения платформы заданным линейной и угловой скоростям.

Таким образом, постановка задачи управления включает три взаимосвязанных этапа. Во-первых, необходимо преобразовать входные команды верхнего уровня, задаваемые в виде $v_{зад}(t)$ и $\omega_{зад}(t)$, в опорные параметры движения ведущих колёс. Во-вторых, необходимо определять фактические линейную и угловую скорости платформы по данным энкодеров ведущих колёс. В-третьих, необходимо формировать такие управляющие воздействия на приводы, которые обеспечивают уменьшение ошибок $e_v(t)$ и $e_\omega(t)$ и, как следствие, воспроизведение требуемого режима движения платформы.

Полученная постановка задачи непосредственно подводит к синтезу закона управления, сочетающего использование математической модели и принципа обратной связи.

2.3. Синтез закона управления

Для решения сформулированной задачи в работе используется комбинированный закон управления, включающий две составляющие: модельную упреждающую составляющую и корректирующую составляющую, формируемую по обратной связи. Применение такой структуры обусловлено тем, что математическая модель позволяет определить номинальный режим работы системы, соответствующий заданному движению платформы, однако не учитывает полностью влияние неидеальности приводов, внешних возмущений, разброса параметров и ошибок измерения. Введение корректирующего контура позволяет компенсировать возникающие отклонения и повысить точность воспроизведения требуемого режима движения [1, 3].

На первом этапе по заданным линейной и угловой скоростям платформы определяются требуемые линейные скорости ведущих колёс на основе кинематической модели дифференциальной платформы [2, 5, 8]:

$$v_{L, \text{зад}}(t) = v_{\text{зад}}(t) - \frac{L \omega_{\text{зад}}(t)}{2}$$
$$v_{R, \text{зад}}(t) = v_{\text{зад}}(t) + \frac{L \omega_{\text{зад}}(t)}{2}$$

На втором этапе полученные значения преобразуются в требуемые угловые скорости вращения колёс [2, 8]:

$$\omega_{L, \text{зад}}(t) = \frac{v_{L, \text{зад}}(t)}{r}$$
$$\omega_{R, \text{зад}}(t) = \frac{v_{R, \text{зад}}(t)}{r}$$

Полученные значения $\omega_{L, \text{зад}}(t)$ и $\omega_{R, \text{зад}}(t)$ задают номинальный режим вращения ведущих колёс, необходимый для реализации требуемого движения платформы. Однако сами по себе эти величины ещё не являются конечным управляющим воздействием, поскольку исполнительные механизмы приводятся в действие напряжением либо эквивалентным ему управляющим сигналом [10].

Для вычисления номинального управляющего воздействия используется статическая модель привода в установившемся режиме. В соответствии с данной моделью требуемое напряжение на двигателе определяется как сумма постоянной составляющей, необходимой для преодоления статических потерь, и линейной составляющей, пропорциональной требуемой угловой скорости вращения. Для левого и правого приводов номинальные напряжения определяются выражениями [10]:

$$U_{L,ff}(t) = k_S \operatorname{sgn}(\omega_{L,зад}(t)) + k_V \omega_{L,зад}(t)$$

$$U_{R,ff}(t) = k_S \operatorname{sgn}(\omega_{R,зад}(t)) + k_V \omega_{R,зад}(t)$$

где k_S — коэффициент статической составляющей напряжения, а k_V — коэффициент пропорциональности между требуемой угловой скоростью и напряжением двигателя в установившемся режиме.

Слагаемые $k_S \operatorname{sgn}(\omega_{L,зад}(t))$ и $k_S \operatorname{sgn}(\omega_{R,зад}(t))$ учитывают необходимость преодоления трения, зоны нечувствительности и других статических потерь в приводах. Слагаемые $k_V \omega_{L,зад}(t)$ и $k_V \omega_{R,зад}(t)$ отражают приближённую линейную зависимость между требуемой угловой скоростью вращения соответствующего колеса и напряжением двигателя в установившемся режиме. Следовательно, величины $U_{L,ff}(t)$ и $U_{R,ff}(t)$ представляют собой номинальные, или упреждающие, управляющие воздействия, рассчитываемые по модели без учёта текущего отклонения фактического движения платформы от заданного.

Однако в реальной системе использование только модельной составляющей, как правило, не обеспечивает требуемой точности регулирования. Это связано с тем, что реальный объект не совпадает полностью с принятой моделью, а также подвержен действию внешних и внутренних возмущений. Поэтому в закон управления вводится корректирующая составляющая, формируемая по ошибкам линейной и угловой скоростей платформы [1, 3]:

$$e_v(t) = v_{зад}(t) - v(t)$$

$$e_{\omega}(t) = \omega_{зад}(t) - \omega(t)$$

В качестве корректирующего звена используются два регулятора структуры ПИД: регулятор линейной скорости платформы и регулятор её угловой скорости. Конкретный режим их работы определяется значениями коэффициентов, выбранными по результатам настройки. Первый регулятор работает по ошибке линейной скорости платформы, второй — по ошибке её угловой скорости. Такой выбор обусловлен тем, что управление в рассматриваемой системе ориентировано на воспроизведение режима движения платформы как единого объекта, а не на независимое регулирование каждого колеса в отдельности.

Корректирующие воздействия определяются выражениями [3]:

$$u_v(t) = K_{p_v} e_v(t) + K_{I_v} \int_0^t e_v(\tau) d\tau + K_{D_v} \frac{de_v(t)}{dt}$$

$$u_{\omega}(t) = K_{p_{\omega}} e_{\omega}(t) + K_{I_{\omega}} \int_0^t e_{\omega}(\tau) d\tau + K_{D_{\omega}} \frac{de_{\omega}(t)}{dt}$$

где $K_{p_v}, K_{I_v}, K_{D_v}$ — коэффициенты ПИД-регулятора линейной скорости, а $K_{p_{\omega}}, K_{I_{\omega}}, K_{D_{\omega}}$ — коэффициенты ПИД-регулятора угловой скорости.

Пропорциональная составляющая обеспечивает реакцию на текущее значение ошибки и способствует её оперативному уменьшению. Интегральная составляющая учитывает накопленную во времени ошибку и позволяет снижать установившееся рассогласование между заданным и фактическим режимом движения. Дифференциальная составляющая реагирует на скорость изменения ошибки и при корректной настройке улучшает характер переходного процесса и уменьшает колебательность системы [3].

Поскольку исполнительными механизмами платформы являются левый и правый приводы, корректирующие воздействия по линейной и угловой скорости должны быть преобразованы в добавочные управляющие воздействия для каждого канала.

Для этого используется следующее смещение:

$$\Delta U_L(t) = u_v(t) - u_\omega(t)$$

$$\Delta U_R(t) = u_v(t) + u_\omega(t)$$

Итоговые управляющие воздействия на левый и правый приводы формируются как сумма модельной и корректирующей составляющих [1, 3]:

$$U_L(t) = U_{L,ff}(t) + \Delta U_L(t)$$

$$U_R(t) = U_{R,ff}(t) + \Delta U_R(t)$$

Или, подставляя выражения для корректирующих воздействий, можно записать

$$U_L(t) = U_{L,ff}(t) + u_v(t) - u_\omega(t)$$

$$U_R(t) = U_{R,ff}(t) + u_v(t) + u_\omega(t)$$

Полученные величины $U_L(t)$ и $U_R(t)$ представляют собой конечный результат работы закона управления и подаются на исполнительные механизмы приводов. Таким образом, синтезированный закон управления реализует последовательную цепочку преобразований: от задания режима движения платформы в терминах линейной и угловой скоростей — к вычислению требуемых линейных, а затем угловых скоростей колёс, к определению номинальных напряжений на приводах и далее к их коррекции по сигналам обратной связи.

Содержательно работа синтезированного закона управления может быть описана следующим образом. По входной команде движения сначала вычисляются требуемые линейные скорости ведущих колёс, после чего они преобразуются в требуемые угловые скорости их вращения. Затем по модели привода определяется напряжение, которое в идеализированном случае должно обеспечить данный режим. После этого на основе данных энкодеров ведущих колёс определяется отклонение фактических линейной и угловой скоростей платформы от требуемых значений, а корректирующие регуляторы формируют воздействия по двум каналам управления. На заключительном этапе эти корректирующие воздействия преобразуются в добавки к левому и правому

приводам и суммируются с модельной составляющей. Тем самым система реализует замкнутый принцип управления, в котором модельная часть задаёт номинальное поведение, а обратная связь обеспечивает коррекцию возникающих отклонений [1, 2, 3, 10].

Следовательно, синтезированный закон управления обеспечивает переход от абстрактной команды движения верхнего уровня к физически реализуемым управляющим воздействиям на приводы мобильной платформы. Использование кинематической модели позволяет определить требуемые параметры движения колёс, модель привода — вычислить номинальные напряжения, а корректирующие регуляторы по линейной и угловой скорости платформы — компенсировать отклонения фактического движения от требуемого режима. В совокупности это формирует теоретическую основу программной реализации алгоритма управления, рассматриваемой в следующей главе.

В практической реализации синтезированного закона управления используются конкретные значения коэффициентов модели привода и коэффициентов корректирующих регуляторов. В рамках настоящей работы данные параметры определялись последовательным изменением коэффициентов с последующим запуском моделирования и анализом поведения замкнутой системы. В качестве основного критерия выбора параметров использовалось уменьшение ошибки регулирования по линейной и угловой скорости при сохранении устойчивого характера переходного процесса и отсутствии нарастающих колебаний [3]. При проверке конфигурации с ненулевой дифференциальной составляющей углового регулятора наблюдалось ухудшение поведения системы. Поэтому для последующих испытаний принята конфигурация с отключённой дифференциальной составляющей обоих регуляторов.

Итоговые значения коэффициентов, использованные в реализованной системе управления, приведены в таблице 1.

Таблица 1 — Значения коэффициентов модели привода и регуляторов

Параметр	Значение	Единица измерения	Назначение
k_s	0,1	В	коэффициент статической составляющей модели привода
k_v	0,232	В·с/рад	коэффициент скоростной составляющей модели привода
$K_{p,v}$	0,8	В·с/м	пропорциональный коэффициент регулятора линейной скорости
$K_{i,v}$	0,4	В/м	интегральный коэффициент регулятора линейной скорости
$K_{d,v}$	0	В·с ² /м	дифференциальный коэффициент регулятора линейной скорости
$K_{p,\omega}$	0,4	В·с/рад	пропорциональный коэффициент регулятора угловой скорости
$K_{i,\omega}$	0,2	В/рад	интегральный коэффициент регулятора угловой скорости
$K_{d,\omega}$	0	В·с ² /рад	дифференциальный коэффициент регулятора угловой скорости

Испытания в среде моделирования проводились на программной реализации системы управления с использованием приведённых значений коэффициентов. Следовательно, результаты, представленные в последующих разделах работы, относятся именно к данному набору параметров системы управления.

2.4. Выводы

В результате построения математической модели и постановки задачи управления установлено, что управление мобильной платформой должно обеспечивать преобразование заданных линейной и угловой скоростей в номинальные параметры движения ведущих колёс с последующим формированием управляющих воздействий на левый и правый приводы. Тем самым задача управления сведена к обеспечению соответствия фактических линейной и угловой скоростей платформы их заданным значениям при

использовании кинематической модели для вычисления номинального режима работы приводов.

В ходе синтеза закона управления показано, что выбранная комбинированная структура системы управления включает модельную упреждающую составляющую и корректирующую составляющую, формируемую по сигналам обратной связи. Модельная часть позволяет по заданным параметрам движения определить требуемые угловые скорости колёс и соответствующие им номинальные напряжения приводов. Корректирующая часть, реализованная на основе регуляторов линейной и угловой скорости платформы, обеспечивает компенсацию отклонений, возникающих вследствие неточности модели и действия внешних возмущений. В экспериментальной конфигурации указанные регуляторы реализованы по ПИ-закону.

Таким образом, в разделе получен закон управления, обеспечивающий переход от команды движения верхнего уровня к физически реализуемым управляющим воздействиям на приводы мобильной платформы.

3. Разработка программы

3.1. Назначение программы

Разрабатываемая программа предназначена для управления ходовой частью четырёхколёсного мобильного робота с дифференциальной схемой привода. Назначение программы состоит в формировании управляющих воздействий на левый и правый приводы ведущих колёс в соответствии с заданными параметрами движения мобильной платформы.

Согласно постановке задачи, сформулированной в главе 2, входными величинами программы являются заданные линейная и угловая скорости платформы. Выходными величинами являются управляющие воздействия на левый и правый приводы, представленные значениями напряжений, подаваемых на двигатели.

Программа должна реализовывать управление по замкнутому контуру. Для организации обратной связи используются данные энкодеров ведущих колёс. По результатам измерения угловых скоростей колёс рассчитываются фактические линейная и угловая скорости платформы. Сравнение заданных и фактических значений позволяет определять ошибки регулирования и формировать корректирующую составляющую управляющего воздействия [1, 2].

В соответствии с разработанным законом управления итоговые воздействия должны формироваться путём объединения модельной и корректирующей составляющих. Модельная составляющая определяет номинальные воздействия, соответствующие заданному режиму движения платформы. Корректирующая составляющая предназначена для уменьшения отклонений фактических линейной и угловой скоростей от заданных значений. Согласно параметрам, выбранным в разделе 2.3, регуляторы линейной и угловой скорости должны работать по ПИ-закону [1, 3, 10].

Поскольку программа предназначена для выполнения на микроконтроллерной платформе STM32, вычислительная логика управления движением должна быть отделена от средств взаимодействия с датчиками и

исполнительными механизмами. Такое разделение позволяет независимо развивать и модифицировать математические модели и алгоритмы управления, не затрагивая платформенно-зависимый код, а также упрощает адаптацию программы к конкретной конфигурации робота и используемому оборудованию [7, 9]. Возможность проверки одного и того же вычислительного контура в среде моделирования и его последующего выполнения на STM32 является следствием выбранной архитектуры.

Таким образом, разрабатываемая программа должна обеспечивать дискретное управление линейной и угловой скоростями мобильной платформы по замкнутому контуру: принимать заданные параметры движения, получать данные обратной связи от энкодеров, рассчитывать фактическое состояние платформы, формировать управляющие воздействия на приводы и передавать их аппаратно-зависимому уровню системы.

3.2. Функциональные требования к программе

Функциональные требования к разрабатываемой программе определяются её назначением, законом управления, полученным в главе 2, и необходимостью выполнения вычислительной логики на микроконтроллерной платформе STM32. Программа должна обеспечивать управление скоростным режимом мобильной платформы с использованием обратной связи по данным энкодеров ведущих колёс.

К разрабатываемой программе предъявляются следующие функциональные требования:

1. Приём команды движения. Программа должна принимать заданные значения линейной и угловой скоростей мобильной платформы.
2. Получение данных обратной связи. Программа должна обеспечивать получение данных от энкодеров левого и правого ведущих колёс, необходимых для определения фактического режима движения платформы.

3. Вычисление фактических скоростей платформы. На основе данных энкодеров программа должна определять фактические линейную и угловую скорости мобильной платформы.
4. Вычисление ошибок регулирования. Программа должна определять рассогласования между заданными и фактическими значениями линейной и угловой скоростей платформы.
5. Формирование номинального управляющего воздействия. Программа должна вычислять номинальные воздействия на левый и правый приводы по заданной команде движения с использованием кинематической модели платформы, её геометрических параметров и модели привода.
6. Формирование корректирующего воздействия. Программа должна вычислять корректирующие составляющие управления по ошибкам линейной и угловой скоростей. В соответствии с параметрами, выбранными в разделе 2.3, корректирующие регуляторы должны работать по ПИ-закону.
7. Формирование результирующих воздействий на приводы. Программа должна объединять номинальную и корректирующую составляющие управления и формировать значения напряжения отдельно для левого и правого приводов ведущих колёс.
8. Дискретный режим выполнения управления. Вычисление управляющих воздействий должно выполняться в повторяющемся дискретном цикле, включающем получение входных данных, формирование текущего состояния платформы, расчёт управления и передачу результата на исполнительный уровень [4].
9. Обеспечение режима остановки. При нулевых заданных значениях линейной и угловой скоростей программа должна формировать управляющие воздействия, соответствующие прекращению движения платформы.
10. Отделение вычислительной логики от средств взаимодействия с устройствами. Алгоритмы формирования управляющих воздействий не должны зависеть от конкретного способа получения данных энкодеров и передачи

напряжений на приводы. Взаимодействие с исполнительным и измерительным уровнем должно осуществляться через слой аппаратной абстракции [12].

11. Выполнение на платформе STM32. Архитектура программы должна обеспечивать включение вычислительной логики управления в состав встраиваемой программы для микроконтроллерной платформы STM32 и её работу при взаимодействии с объектом управления [6, 19].

Перечисленные требования определяют границы разрабатываемой программы. Основным объектом регулирования являются линейная и угловая скорости мобильной платформы. Управление её положением в глобальной системе координат в состав требований не включается. Поэтому анализ траекторий при последующих испытаниях используется для оценки поведения скоростного контура, а не как подтверждение реализации отдельного позиционного управления.

Состав программных компонентов, предназначенных для выполнения указанных требований, и порядок их взаимодействия рассматриваются в следующем разделе.

3.3. Архитектура программы

Для реализации требований, сформулированных в разделе 3.2, принимается архитектура программы, основанная на разделении организации управления, реализации закона управления и средств взаимодействия с измерительным и исполнительным уровнем. Такое построение позволяет выделить компоненты, непосредственно определяющие формирование управляющего воздействия, и отделить их от компонентов, обеспечивающих получение измерений и передачу рассчитанных воздействий приводам [7, 9, 12].

В настоящем разделе рассматривается архитектура рабочего контура управления, то есть совокупность программных компонентов, непосредственно участвующих в формировании воздействий на левый и правый приводы. В соответствии с принятыми требованиями данный контур должен обеспечивать приём команды движения, получение обратной связи по данным энкодеров

ведущих колёс, формирование текущего состояния платформы, вычисление результирующего воздействия и его передачу исполнительному уровню.

В составе архитектуры рабочего контура выделяются три основные подсистемы:

1. Подсистема организации управления и представления состояния платформы.
2. Подсистема реализации закона управления.
3. Подсистема аппаратной абстракции.

Центральными компонентами принятой архитектуры являются `RobotController` и `Robot`. Компонент `RobotController` предназначен для организации формирования управляющего воздействия по заданной команде движения и текущему состоянию платформы. Компонент `Robot` предназначен для представления управляемой платформы на программном уровне, формирования её текущего состояния по данным измерительной части и передачи рассчитанных воздействий исполнительной части.

Подсистема реализации закона управления включает модельный компонент `FFModel` и регуляторы линейной и угловой скорости. Подсистема аппаратной абстракции включает предметные компоненты `Encoder` и `Motor`, а также интерфейсы `HAL::IEncoderHAL` и `HAL::IMotorHAL`, через которые осуществляется связь рабочего контура с конкретными средствами получения измерений и передачи воздействий [12]. Общая структура архитектуры рабочего контура управления представлена на рисунке 2.

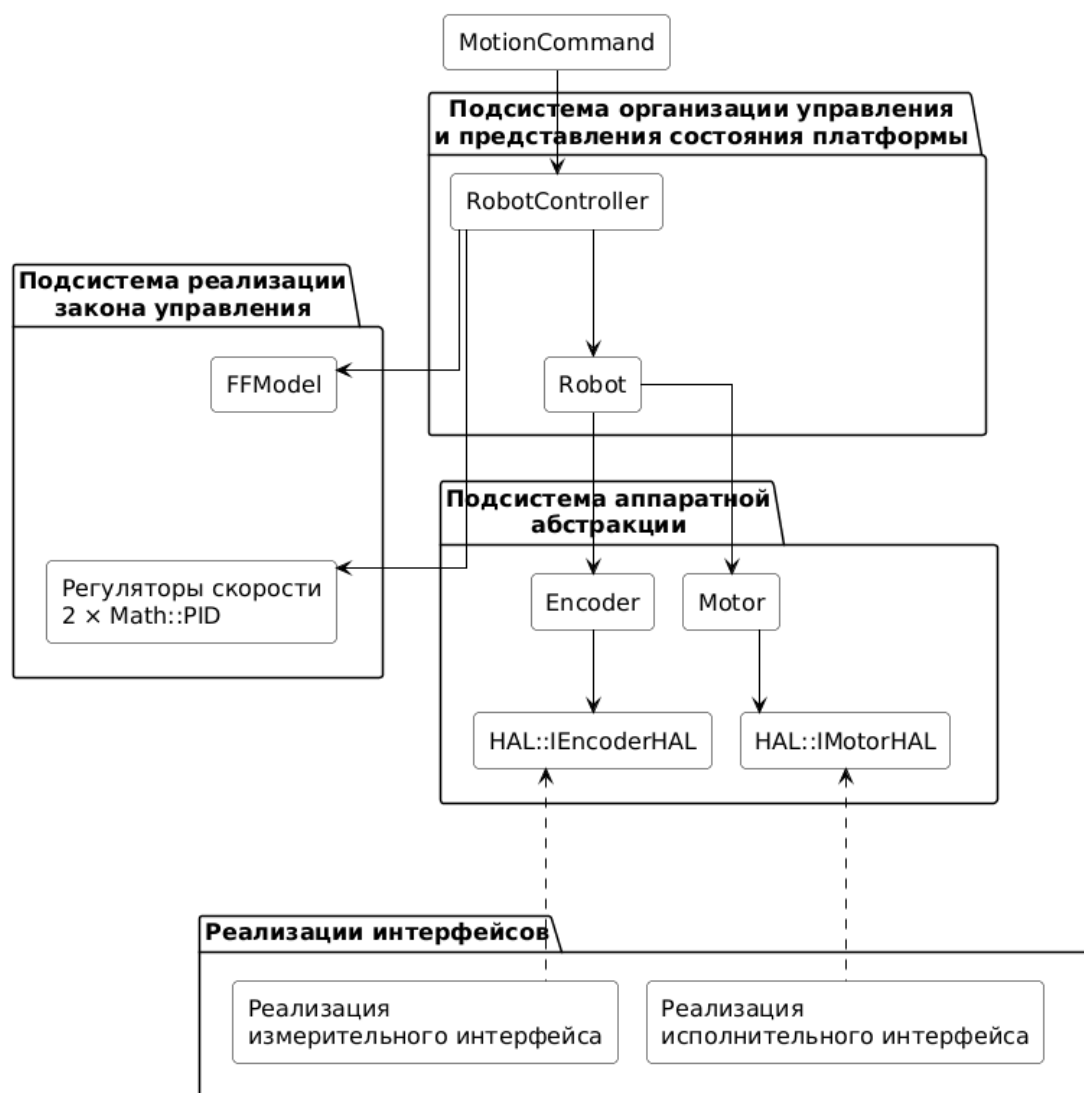


Рисунок 2 — Общая структура архитектуры рабочего контура управления

Как показано на рисунке 2, архитектура рабочего контура имеет иерархическое построение. Подсистема организации управления и представления состояния платформы содержит компоненты **RobotController** и **Robot**, связывающие команду движения, вычисление воздействия и программное представление платформы. Компонент **RobotController** использует подсистему реализации закона управления, включающую модельную и корректирующую части. Компонент **Robot** взаимодействует с подсистемой аппаратной абстракции, через которую выполняется связь с измерительными и исполнительными средствами.

Такое построение соответствует требованиям к программе: закон управления выделен в самостоятельную подсистему, обратная связь рабочего контура формируется по данным энкодеров, а зависимость от конкретного способа получения измерений и передачи воздействий локализуется на уровне аппаратной абстракции.

3.3.1. Подсистема организации управления и представления состояния платформы

Подсистема организации управления и представления состояния платформы предназначена для связи входной команды движения, текущего состояния объекта управления, компонентов реализации закона управления и результата вычисления воздействия. Основными компонентами данной подсистемы являются `RobotController` и `Robot`.

Компонент `RobotController` предназначен для формирования управляющего воздействия в соответствии с законом управления, разработанным в разделе 2.3. Входная команда движения представляется структурой `MotionCommand`, содержащей заданные линейную и угловую скорости платформы. Результат вычисления представляется структурой `ControlEffort`, содержащей значения напряжения для левого и правого приводов.

Для формирования корректирующей составляющей управления компоненту `RobotController` требуется информация о фактическом режиме движения платформы. Данная информация представляется структурой `RobotState`, содержащей фактические линейную и угловую скорости платформы. В соответствии с требованиями программы указанные скорости определяются по данным энкодеров ведущих колёс.

Компонент `Robot` предназначен для представления мобильной платформы в составе рабочего контура программы. Через данный компонент обеспечивается связь текущего состояния платформы с данными измерительной части и связь

рассчитанного управляющего воздействия с исполнительная частью. Тем самым Robot образует границу между компонентами организации управления и подсистемой аппаратной абстракции.

Принятое разделение ответственности позволяет исключить непосредственную зависимость компонента RobotController от конкретного способа получения измерений и передачи воздействий приводам. Компонент управления работает со структурами MotionCommand, RobotState и ControlEffort, тогда как связь этих структур с измерительными и исполнительными средствами обеспечивается через компонент Robot.

3.3.2. Подсистема реализации закона управления

Подсистема реализации закона управления предназначена для формирования воздействий на левый и правый приводы в соответствии с комбинированным законом управления, принятым в главе 2. В её составе выделяются модельный компонент FFModel и регуляторы линейной и угловой скорости.

Компонент FFModel предназначен для формирования модельной составляющей управляющего воздействия. На основе команды движения, геометрических параметров платформы и коэффициентов модели привода он должен вычислять номинальные значения напряжения для левого и правого приводов. Данные значения соответствуют требуемому режиму движения при принятой модели объекта управления.

Корректирующая составляющая управления формируется двумя регуляторами на основе класса Math::PID. Первый регулятор предназначен для обработки ошибки линейной скорости платформы, второй — для обработки ошибки угловой скорости. В соответствии с параметрами, выбранными в разделе 2.3, дифференциальные коэффициенты регуляторов принимаются равными нулю, поэтому в рассматриваемой конфигурации корректирующая составляющая формируется по ПИ-закону.

Результирующее управляющее воздействие формируется путём объединения модельной и корректирующей составляющих. Полученный результат используется компонентом `RobotController` для формирования напряжений левого и правого приводов, представленных структурой `ControlEffort`.

Выделение реализации закона управления в самостоятельную подсистему позволяет явно связать программную архитектуру с математической моделью и структурой регулирования, разработанными в главе 2. При этом вопросы порядка выполнения расчётов и обработки команд относятся к программной реализации цикла управления и рассматриваются в следующем разделе.

3.3.3. Подсистема аппаратной абстракции

Подсистема аппаратной абстракции предназначена для отделения компонентов управления от конкретного способа получения данных измерительной части и передачи воздействий исполнительным механизмам. В рабочем контуре программы для этого используются предметные компоненты `Encoder` и `Motor`, а также интерфейсы `HAL::IEncoderHAL` и `HAL::IMotorHAL` [12].

Компонент `Encoder` предназначен для предоставления данных о движении соответствующего ведущего колеса. Доступ к источнику этих данных осуществляется через интерфейс `HAL::IEncoderHAL`. На основе данных двух объектов `Encoder` компонент `Robot` формирует фактические линейную и угловую скорости платформы, используемые в контуре обратной связи.

Компонент `Motor` предназначен для передачи рассчитанного управляющего напряжения соответствующему приводу. Передача значения воздействия исполнительной части осуществляется через интерфейс `HAL::IMotorHAL`.

Применение интерфейсов аппаратной абстракции позволяет отделить структуру и логику рабочего контура от конкретного способа взаимодействия с

измерительными и исполнительными средствами. Компоненты верхнего уровня зависят от установленных интерфейсов, тогда как особенности конкретной реализации внешнего взаимодействия локализуются за границей данных интерфейсов.

Таким образом, подсистема аппаратной абстракции обеспечивает архитектурное разделение закона управления и средств связи с объектом управления. За счёт этого изменение способа получения данных энкодеров либо передачи напряжений приводам не требует изменения структуры подсистемы реализации закона управления.

3.3.4. Выводы

В результате архитектурного проектирования определён состав компонентов рабочего контура управления ходовой частью мобильной платформы и распределена их ответственность. В основу архитектуры положено разделение организации управления и представления состояния платформы, реализации закона управления и взаимодействия с измерительной и исполнительной частью через аппаратную абстракцию.

Центральными компонентами принятой архитектуры являются `RobotController` и `Robot`. Компонент `RobotController` организует формирование управляющего воздействия, используя подсистему реализации закона управления, включающую `FFModel` и регуляторы линейной и угловой скорости. Компонент `Robot` обеспечивает связь формализованного состояния и рассчитанных воздействий с объектами `Encoder` и `Motor`.

Обратная связь рабочего контура формируется по данным энкодеров ведущих колёс. Корректирующая часть управления реализуется регуляторами, настроенными по ПИ-закону, а результирующее воздействие представляется напряжениями левого и правого приводов в структуре `ControlEffort`.

Применение интерфейсов аппаратной абстракции `HAL::IEncoderHAL` и `HAL::IMotorHAL` позволяет отделить подсистему реализации закона управления

от конкретных средств взаимодействия с измерительной и исполнительной частью. Таким образом, принятая архитектура соответствует требованиям, сформулированным в разделе 3.2, и задаёт основу для программной реализации дискретного цикла управления, рассматриваемой в следующем разделе.

3.4. Программная реализация цикла управления

В данном разделе рассматривается реализация дискретного шага управления на уровне программных операций и методов компонентов рабочего контура. В рамках одного шага выполняются обновление измерительных данных, формирование текущего состояния платформы, вычисление управляющего воздействия и передача рассчитанных напряжений приводам [4]. Далее описываются только те процедуры реализации, которые непосредственно участвуют в формировании и применении проверяемого управляющего воздействия.

3.4.1. Общий шаг управления

На верхнем уровне программного цикла один шаг управления состоит из трёх последовательно выполняемых операций:

1. Обновление измерительных данных.
2. Вычисление управляющего воздействия.
3. Передача вычисленного воздействия приводам.

В программной реализации общий порядок выполнения одного шага управления верхнеуровнево может быть представлен следующей последовательностью вызовов, изображенной на рисунке 3.

```
1 robot.UpdateSensors();  
2  
3 const Robot::ControlEffort effort =  
4     controller.GetAdjustedControlEffort(cmd, dt);  
5  
6 robot.TransferToNewState(effort, dt);
```

Рисунок 3 — Общий порядок выполнения одного шага управления

Вызов `UpdateSensors()` обеспечивает обновление измерительных данных, доступных компоненту `Robot`. В рабочем контуре, рассматриваемом в данной работе, при последующем формировании состояния используются данные левого и правого энкодеров ведущих колёс.

Метод `RobotController::GetAdjustedControlEffort(...)` выполняет вычислительную часть шага управления. На его вход передаются команда движения `MotionCommand` и шаг дискретизации Δt . Внутри метода запрашивается текущее состояние платформы, формируются модельная и корректирующая составляющие управления и создаётся структура `ControlEffort`, содержащая рассчитанные напряжения приводов. Это соответствует реализации метода: он получает состояние через `FetchCurrentRobotState(dt)`, вызывает `FFModel` и два регулятора `Math::PID`, затем формирует напряжения левого и правого каналов.

Метод `Robot::TransferToNewState(...)` связывает результат вычисления с исполнительной частью программы. Через объекты `Motor` рассчитанные напряжения передаются подсистеме аппаратной абстракции для последующего применения к приводам. Последовательность выполнения одного шага управления представлена на рисунке 4.

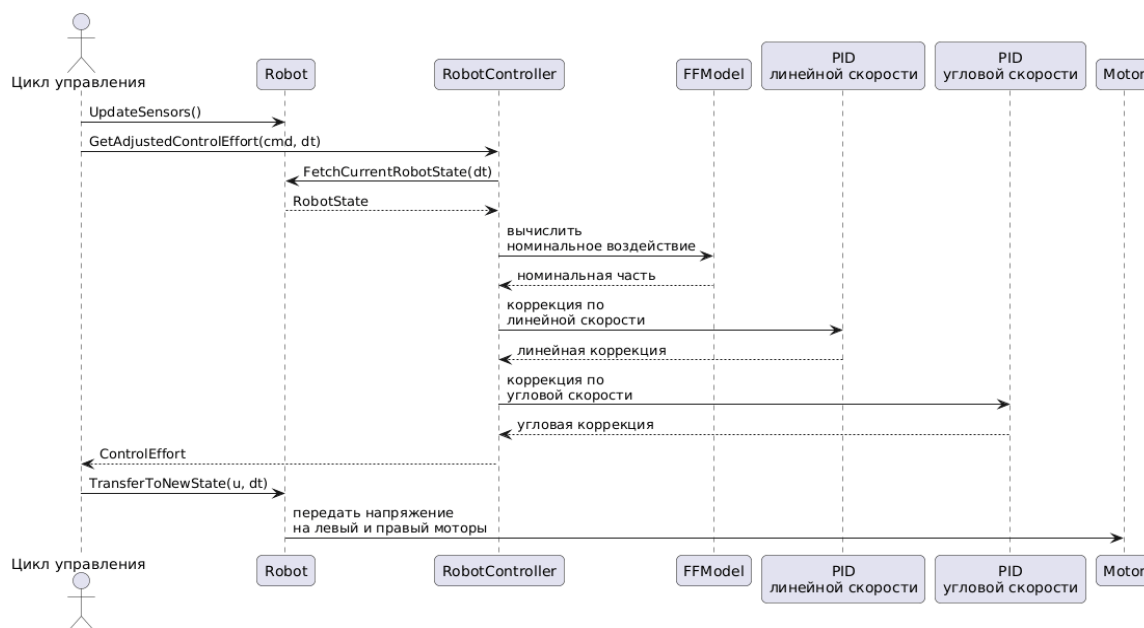


Рисунок 4 — Последовательность выполнения одного шага управления

Как показано на рисунке 4, компонент `RobotController` выполняет вычисление управляющего воздействия на основе команды движения и состояния, полученного от компонента `Robot`. Передача рассчитанного результата приводам выполняется отдельно, после завершения вычислительного этапа. Такое построение соответствует принятому в архитектуре разделению формирования управления и взаимодействия с исполнительной частью.

3.4.2. Формирование текущего скоростного состояния платформы

Текущее состояние платформы, используемое при вычислении управляющего воздействия, формируется методом `Robot::FetchCurrentRobotState(...)`. Исходными данными для данного этапа являются линейные скорости левого и правого ведущих колёс, получаемые через объекты `Encoder`.

До вызова метода `Robot::FetchCurrentRobotState(...)` в общем шаге управления выполняется метод `Robot::UpdateSensors()`, инициирующий обновление состояния энкодеров. Компонент `Encoder` получает от соответствующей реализации интерфейса `HAL::IEncoderHAL` текущее значение линейной скорости колеса и сохраняет его для последующего использования

компонентом Robot. Тем самым преобразование первичного измерения энкодера в линейную скорость колеса локализуется в реализации аппаратной абстракции, а основная логика управления работает уже с физическими величинами скорости.

Перед вычислением скоростных параметров платформы метод `FetchCurrentRobotState(...)` проверяет допустимость шага дискретизации, работоспособность энкодеров и корректность полученных значений скоростей. При невозможности сформировать допустимое состояние нормальное обновление обратной связи не выполняется; обработка соответствующих состояний рассматривается в подразделе 3.4.4.

В общем случае линейная скорость ведущего колеса может быть определена по приращению его углового положения за интервал дискретизации:

$$v_i = r \cdot \frac{\Delta \varphi_i}{\Delta t}$$

где v_i — линейная скорость соответствующего колеса; r — радиус колеса; $\Delta \varphi_i$ — изменение углового положения колеса за интервал Δt .

При представлении измерения в виде приращения отсчётов энкодера данное выражение принимает вид [2]:

$$v_i = \frac{2 \cdot \pi \cdot r}{N_{\text{rev}}} \cdot \frac{\Delta N_i}{\Delta t}$$

где N_{rev} — число отсчётов энкодера на один оборот колеса; ΔN_i — изменение числа отсчётов соответствующего энкодера за рассматриваемый интервал времени.

После получения линейных скоростей колёс компонент Robot формирует скоростное состояние платформы. Пусть v_L и v_R — линейные скорости левого и правого ведущих колёс соответственно, а B — расстояние между ведущими колёсами. Тогда фактическая линейная скорость платформы определяется выражением:

$$v = \frac{v_L + v_R}{2}$$

Фактическая угловая скорость платформы определяется выражением:

$$\omega = \frac{v_R - v_L}{B}$$

Порядок формирования текущего скоростного состояния платформы представлен на рисунке 5.

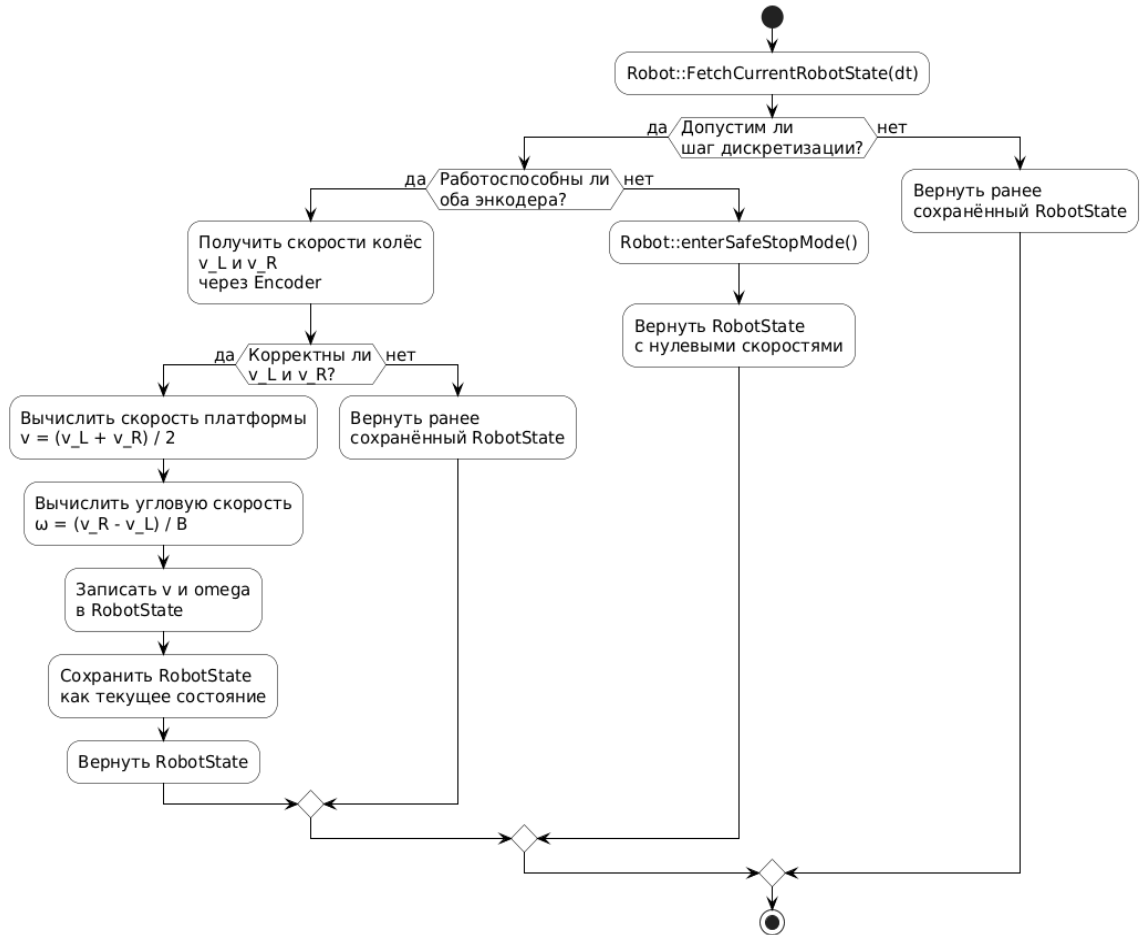


Рисунок 5 — Блок-схема формирования текущего скоростного состояния платформы

Как показано на рисунке 5, компонент Robot получает через объекты Encoder значения линейных скоростей ведущих колёс v_L и v_R . На их основе методом `FetchCurrentRobotState(...)` вычисляются фактические линейная и угловая скорости платформы.

Полученные значения записываются в структуру `RobotState` и передаются компоненту `RobotController` для вычисления ошибок скоростного управления. В используемой конфигурации значение скорости, предоставляемое объектом

Encoder, соответствует текущему значению, полученному через интерфейс `HAL::IEncoderHAL`.

Таким образом, обратная связь рабочего контура формируется в два этапа: сначала первичные данные энкодеров преобразуются в линейные скорости ведущих колёс, затем по кинематическим соотношениям дифференциальной платформы вычисляются текущие линейная и угловая скорости мобильной платформы.

3.4.3. Вычисление управляющего воздействия

Вычисление управляющего воздействия выполняется методом `RobotController::GetAdjustedControlEffort(...)`. На вход метода поступают команда движения `MotionCommand`, содержащая заданные линейную и угловую скорости платформы, и шаг дискретизации Δt . В результате расчёта формируются значения напряжения для левого и правого приводов, представляемые структурой `ControlEffort`.

Перед выполнением основного расчёта метод проверяет возможность продолжения нормального управления. При недопустимо большом значении шага дискретизации либо при уже активированном безопасном режиме формируется нулевое воздействие, а состояния регуляторов сбрасываются. При недопустимо малом значении шага дискретизации возвращается последнее сохранённое допустимое воздействие. Подробное описание реакции системы на недопустимые состояния и передачи результата исполнительной части приводится в подразделе 3.4.4.

При допустимых условиях вычисления компонент `RobotController` получает текущее скоростное состояние платформы посредством вызова `Robot::FetchCurrentRobotState(...)`. Полученная структура `RobotState` содержит фактические линейную и угловую скорости платформы, рассчитанные по данным энкодеров в соответствии с процедурой, рассмотренной в подразделе 3.4.2.

Заданная команда движения предварительно обрабатывается методом `VelocityRamp(...)`. Данный метод ограничивает изменение задаваемых линейной и угловой скоростей между соседними шагами дискретного цикла. В результате формируется обработанная команда, значения которой используются при расчёте модельной и корректирующей составляющих управляющего воздействия.

Модельная составляющая формируется компонентом `FFModel`. На основе обработанной команды движения компонент определяет требуемые линейные скорости левого и правого ведущих колёс, преобразует их в соответствующие угловые скорости и вычисляет номинальные значения напряжений приводов по принятой модели двигателя. Поскольку математические соотношения модельной составляющей были получены в разделе 2.3, в настоящем подразделе они повторно не выводятся. Результат работы `FFModel` обозначим как $U_{L, \text{мод}}$ и $U_{R, \text{мод}}$.

Корректирующая составляющая управления формируется двумя регуляторами на основе класса `Math::PID`. Первый регулятор используется для канала линейной скорости, второй — для канала угловой скорости платформы. В рассматриваемой конфигурации дифференциальные коэффициенты обоих регуляторов равны нулю, вследствие чего корректирующая составляющая фактически вычисляется по ПИ-закону.

Ошибки регулирования определяются как разность между значениями обработанной команды движения и текущими скоростями платформы:

$$e_v = \tilde{v} - v$$

$$e_\omega = \tilde{\omega} - \omega$$

где v и ω — заданные линейная и угловая скорости после обработки методом `VelocityRamp(...)`.

Выход регулятора линейной скорости формирует составляющую U_v , одинаково изменяющую воздействие на оба привода. Выход регулятора угловой скорости формирует составляющую U_ω , изменяющую разность воздействий

между левым и правым приводами. В программной реализации корректирующие напряжения определяются выражениями [2, 8]:

$$U_{L,кор} = U_v - U_{\omega}$$

$$U_{R,кор} = U_v + U_{\omega}$$

Полное расчётное воздействие для каждого привода формируется суммированием модельной и корректирующей составляющих [1, 3, 10]:

$$U_L = U_{L,мод} + U_{L,кор}$$

$$U_R = U_{R,мод} + U_{R,кор}$$

Полученные значения U_L и U_R образуют расчётное управляющее воздействие для левого и правого приводов. В программной реализации эти значения записываются в структуру `ControlEffort`, которая далее передаётся на этап применения воздействия. Порядок вычисления управляющего воздействия методом `RobotController::GetAdjustedControlEffort(...)` представлен на рисунке 6.

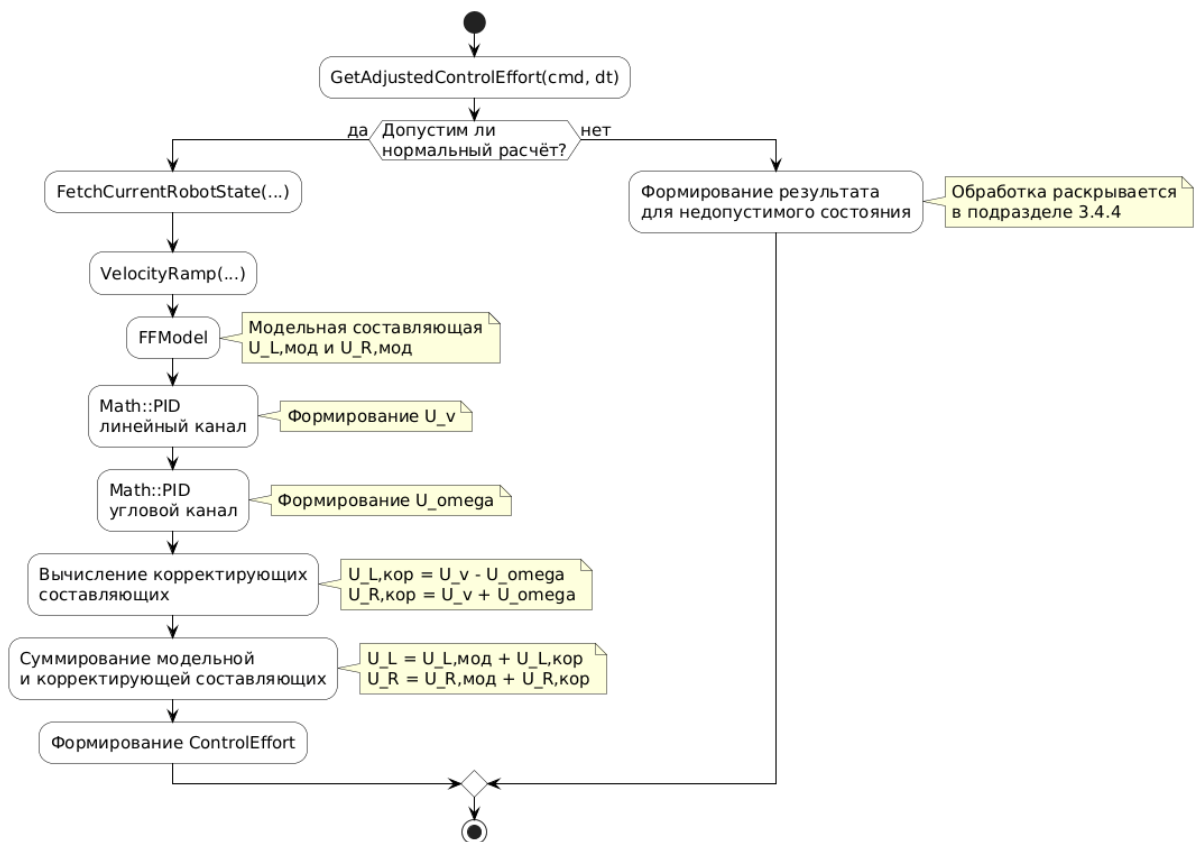


Рисунок 6 — Блок-схема вычисления управляющего воздействия

Как показано на рисунке 6, при допустимых условиях выполнения расчёта метод `GetAdjustedControlEffort(...)` получает текущее скоростное состояние платформы, обрабатывает задающую команду и формирует управляющее воздействие как сумму модельной и корректирующей составляющих. Модельная составляющая задаёт номинальные напряжения приводов, соответствующие требуемому режиму движения, а корректирующая составляющая изменяет их по ошибкам линейной и угловой скоростей платформы.

Таким образом, программная реализация вычисления управляющего воздействия соответствует комбинированному закону управления, разработанному в главе 2. Обработка результата вычисления, его передача исполнительной части и реакция системы на недопустимые состояния рассматриваются в следующем подразделе.

3.4.4. Передача управляющего воздействия и обработка недопустимых состояний

Результатом вычислительного этапа, рассмотренного в подразделе 3.4.3, является структура `ControlEffort`, содержащая рассчитанные значения напряжения для левого и правого приводов. Передача данного воздействия исполнительной части инициируется методом `Robot::TransferToNewState(...)`.

При штатном состоянии рабочего контура компонент `Robot` передаёт значения управляющего воздействия объектам `Motor`, соответствующим левому и правому приводам. Объекты `Motor`, в свою очередь, обеспечивают взаимодействие с исполнительной частью через интерфейс `HAL::IMotorHAL`. Тем самым способ подачи воздействия на приводы остаётся отделённым от алгоритма его вычисления.

До применения нового управляющего воздействия компонент `Robot` проверяет возможность продолжения штатной работы исполнительной части.

Если система уже находится в безопасном режиме, новое воздействие не передаётся приводам. При обнаружении недоступности одного из объектов Motor либо при срабатывании предусмотренной в методе проверки допустимости воздействия вызывается метод `enterSafeStopMode()`. Данный метод устанавливает внутренний признак безопасного режима и завершает остановку исполнительной части передачей нулевых значений напряжения на левый и правый приводы.

Переход в безопасный режим может быть инициирован также на этапе формирования текущего скоростного состояния платформы. Если один из энкодеров недоступен, метод `Robot::FetchCurrentRobotState(...)` вызывает `enterSafeStopMode()` и возвращает структуру состояния с нулевыми значениями скоростей. При недопустимом значении шага дискретизации либо при некорректных числовых значениях скоростей колёс обновление состояния не выполняется, а для дальнейшей обработки возвращается ранее сохранённое состояние платформы.

Обработка недопустимых состояний предусмотрена и в методе `RobotController::GetAdjustedControlEffort(...)`. При превышении допустимого для контроллера значения шага дискретизации либо при наличии уже установленного безопасного режима компонент `RobotController` прекращает нормальное формирование воздействия, сбрасывает состояния регуляторов и возвращает нулевую структуру `ControlEffort`. При слишком малом значении шага дискретизации новое вычисление воздействия не выполняется, а возвращается последнее сохранённое допустимое воздействие.

Таким образом, обработка недопустимых состояний распределена между компонентами рабочего контура в соответствии с их ответственностью. Компонент `RobotController` прекращает нормальный расчёт управляющего воздействия при нарушении условий вычислительного шага либо при активированном безопасном режиме. Компонент `Robot` контролирует возможность формирования скоростного состояния и применения рассчитанного

воздействия исполнительской части. При выявлении отказа измерительного или исполнительского пути система переводится в безопасный режим, при котором применение штатных управляющих воздействий прекращается.

3.4.5. Выводы

В результате программной реализации сформирован дискретный рабочий цикл управления ходовой частью мобильной платформы. В пределах одного шага выполняются обновление измерительных данных, формирование текущего скоростного состояния платформы, вычисление управляющего воздействия и передача результата исполнительской части.

Обратная связь рабочего контура формируется по данным энкодеров ведущих колёс. На основе линейных скоростей левого и правого колёс методом `Robot::FetchCurrentRobotState(...)` вычисляются текущие линейная и угловая скорости платформы, записываемые в структуру `RobotState` и используемые при регулировании.

Вычисление управляющего воздействия выполняется методом `RobotController::GetAdjustedControlEffort(...)`. Модельная составляющая формируется компонентом `FFModel`, а корректирующая составляющая — двумя регуляторами на основе класса `Math::PID`, настроенными в рассматриваемой конфигурации по ПИ-закону. Результатом вычисления является структура `ControlEffort`, содержащая значения воздействия для левого и правого приводов.

Передача рассчитанного воздействия исполнительской части выполняется через компонент `Robot`, объекты `Motor` и интерфейс `HAL::IMotorHAL`. В реализации предусмотрена обработка недопустимых состояний, при которых штатное вычисление либо применение нового управляющего воздействия прекращается, а в соответствующих ветвях выполняется перевод системы в безопасный режим.

Таким образом, реализованный программный цикл соответствует архитектуре, принятой в разделе 3.3, и закону управления, разработанному в

главе 2. Проверка работы данного цикла в составе полного контура управления рассматривается в последующих подразделах.

3.5. Тестирование

Тестирование разработанной программы выполняется с целью подтверждения корректности компонентов рабочего контура управления и оценки его функционирования при воспроизведении заданных режимов движения мобильной платформы. Проверке подлежат компоненты, непосредственно участвующие в получении энкодерной обратной связи, формировании управляющего воздействия и его передаче приводам.

В рамках настоящего раздела применяются два вида проверки. Модульное тестирование предназначено для изолированной проверки программных компонентов рабочего контура в контролируемых условиях. Тестирование в среде моделирования предназначено для оценки совместной работы указанных компонентов в составе замкнутого контура управления при взаимодействии с виртуальной моделью мобильной платформы [7, 9].

Такое построение проверки позволяет сначала подтвердить корректность отдельных программных операций, а затем исследовать поведение системы при выполнении заданных режимов движения. Аппаратно-программная проверка выполнения вычислительной части управления на платформе STM32 рассматривается отдельно в главе 4.

3.5.1. Модульное тестирование

Модульное тестирование предназначено для изолированной проверки программных компонентов, непосредственно участвующих в формировании и применении управляющего воздействия рабочего контура. В соответствии с границами, установленными в разделах 3.3 и 3.4, в настоящем анализе рассматриваются компоненты получения энкодерной обратной связи, реализации закона управления и передачи рассчитанного воздействия приводам.

Для выполнения модульной проверки используется библиотека GoogleTest. Тестовая конфигурация оформлена как отдельный исполняемый

модуль `test_robot`, собираемый для выполнения в вычислительной среде общего назначения. Взаимодействие компонентов с измерительной и исполнительской частью при этом заменяется имитационными реализациями интерфейсов аппаратной абстракции. Такой подход позволяет задавать контролируемые входные данные и проверять результаты вычислений без использования модели мобильной платформы и целевой микроконтроллерной платформы [12, 17].

В рамках рабочего контура управления модульной проверке подлежат следующие компоненты:

1. `Encoder` — получение линейной скорости ведущего колеса через интерфейс `HAL::IEncoderHAL` и обработка состояний доступности измерительного канала.
2. `FFModel` — формирование модельной составляющей управляющего воздействия по заданной команде движения.
3. `Math::PID` — формирование корректирующей составляющей по ошибке регулируемой величины.
4. `RobotController` — объединение модельной и корректирующей составляющих в результирующее воздействие на левый и правый приводы.
5. `Robot` — формирование текущего скоростного состояния платформы по данным двух энкодеров и передача рассчитанного воздействия исполнительской части.
6. `Motor` — взаимодействие с исполнительным интерфейсом при подаче управляющего напряжения.

В исходном проекте предусмотрены также тесты дополнительных компонентов, включая модуль оценки состояния, инерциальный измерительный канал и одометрическую подсистему. Поскольку данные компоненты не определяют значения линейной и угловой скоростей, используемые регуляторами в рассматриваемом рабочем контуре, их испытания не включаются

в основной анализ результатов настоящей работы. Состав модульной проверки компонентов рабочего контура представлен в таблице 2.

Таблица 2 — Состав модульной проверки компонентов рабочего контура управления

Компонент	Количество тестов	Проверяемые аспекты
Motor	6	передача и изменение задаваемого воздействия, ограничение значения напряжения, поведение при нулевом шаге времени
Math::PID	8	реакция на изменение задания, ограничение выхода, обратная коррекция интегральной составляющей, реакция на возмущение
Encoder	5	получение скорости, обработка недоступности измерительного канала и восстановление после отказа
FFModel	6	сценарии формирования модельной составляющей и учёта коэффициентов модели привода
Robot	5	формирование скоростного состояния платформы, передача воздействия и безопасная остановка
RobotController	2	формирование результирующего воздействия и поведение при нулевой команде

Таким образом, для компонентов рабочего контура в составе тестовой конфигурации объявлено 32 тестовых случая. Данный набор охватывает получение данных обратной связи, вычисление модельной и корректирующей составляющих, формирование итогового воздействия и применение результата к исполнительной части программы.

3.5.2. Тестирование в среде моделирования

Тестирование рабочего контура управления выполнено в среде моделирования Webots по схеме программного моделирования в контуре. В данной конфигурации разработанная вычислительная логика выполняется в

среде общего назначения, а объект управления представлен виртуальной моделью мобильной платформы [13]. Взаимодействие программы с энкодерами ведущих колёс и приводами модели осуществляется через реализации интерфейсов аппаратной абстракции, вследствие чего состав и логика вычислительного контура при проведении испытаний не изменяются [15, 16].

В отличие от модульной проверки, испытания в среде моделирования предназначены для оценки совместной работы компонентов рабочего контура при воспроизведении заданных режимов движения. В ходе моделирования выполняются получение энкодерной обратной связи, формирование текущего скоростного состояния платформы, вычисление управляющего воздействия и передача рассчитанных напряжений приводам виртуальной модели.

Для количественного анализа результатов в контроллере моделирования предусмотрено формирование журнала телеметрии. На его основе оцениваются заданные и фактические параметры движения, отклонения зарегистрированных скоростей от заданных сценарием значений, рассчитанные управляющие воздействия и характер движения платформы по рассматриваемым траекториям. Состав телеметрии и используемые показатели качества управления рассматриваются далее в подразделе, посвящённом методике экспериментальной оценки.

3.5.2.1. Интеграция рабочего контура управления со средой моделирования

Интеграция рабочего контура управления с виртуальной моделью мобильной платформы реализована через подсистему аппаратной абстракции. В испытательной конфигурации используются реализации интерфейсов HAL::IEncoderHAL и HAL::IMotorHAL, взаимодействующие с программным интерфейсом среды Webots. Благодаря этому вычислительная часть программы сохраняет состав и порядок работы, рассмотренные в разделах 3.3 и 3.4, а изменяется только способ взаимодействия с измерительной и исполнительной частью [15, 16].

В контроллере моделирования на основе реализаций интерфейсов аппаратной абстракции создаются объекты Encoder и Motor, компонент Robot, модельный компонент FFModel, два регулятора на основе класса Math::PID, настроенные по ПИ-закону, и компонент RobotController. При выполнении шага моделирования данные о движении ведущих колёс через объекты Encoder поступают компоненту Robot и используются для формирования текущего скоростного состояния платформы. Рассчитанные значения напряжений через объекты Motor передаются виртуальным приводам. Схема интеграции рабочего контура управления со средой моделирования представлена на рисунке 7.

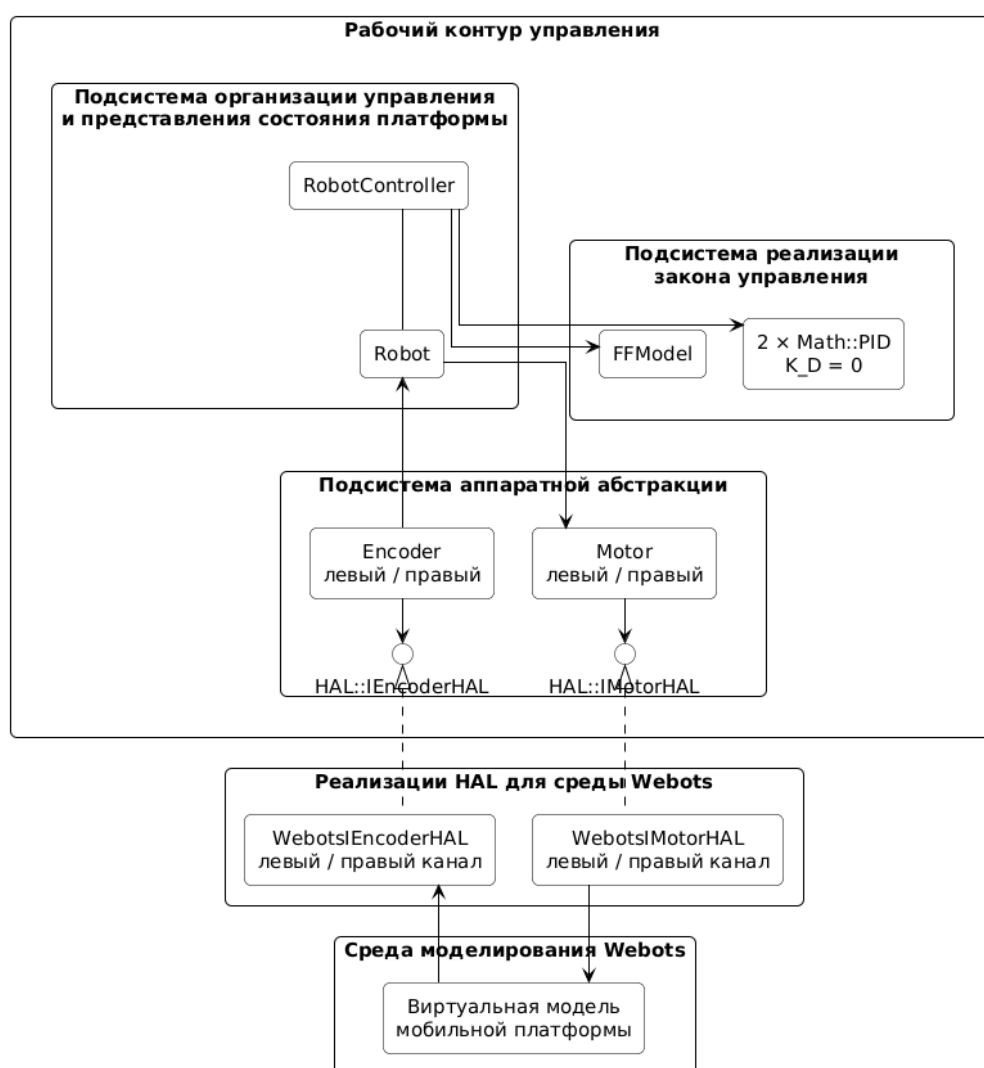


Рисунок 7 — Схема интеграции рабочего контура управления со средой моделирования

Как показано на рисунке 7, замкнутый контур управления формируется передачей данных о движении ведущих колёс из виртуальной модели в разработанную программу и обратной передачей рассчитанных напряжений приводам модели. При этом среда Webots представляет объект управления и обеспечивает программный доступ к его измерительным и исполнительным средствам, а вычисление управляющего воздействия выполняется компонентами рабочего контура без изменения принятой структуры алгоритма.

Таким образом, испытательная конфигурация позволяет проверить совместную работу разработанных компонентов в составе замкнутого контура управления, сохраняя отделение алгоритма расчёта воздействия от среды моделирования. Особенности формирования телеметрии и показатели, используемые для количественной оценки результатов, рассматриваются в следующем подразделе.

3.5.2.2. Методика экспериментальной оценки

Оценка результатов испытаний в среде моделирования выполняется на основе телеметрии, формируемой при каждом шаге заданного сценария движения. В журнал записываются время моделирования, заданные сценарием линейная и угловая скорости платформы, зарегистрированные фактические значения указанных скоростей, отклонения фактических значений от задания, рассчитанные управляющие воздействия для левого и правого приводов, а также координаты положения платформы.

Фактические линейная и угловая скорости платформы фиксируются по структуре RobotState, формируемой рабочим контуром управления на основе данных энкодеров ведущих колёс. Координаты положения платформы, используемые для построения траектории и вычисления ошибки конечного положения, регистрируются с помощью модуля GPS среды Webots [16]. Данные GPS не участвуют в формировании обратной связи и используются только для геометрической оценки результатов движения. Рассчитанные управляющие воздействия для левого и правого приводов соответствуют значениям,

сформированным компонентом RobotController в структуре ControlEffort. Структура формирования телеметрии и расчёта показателей качества представлена на рисунке 8.

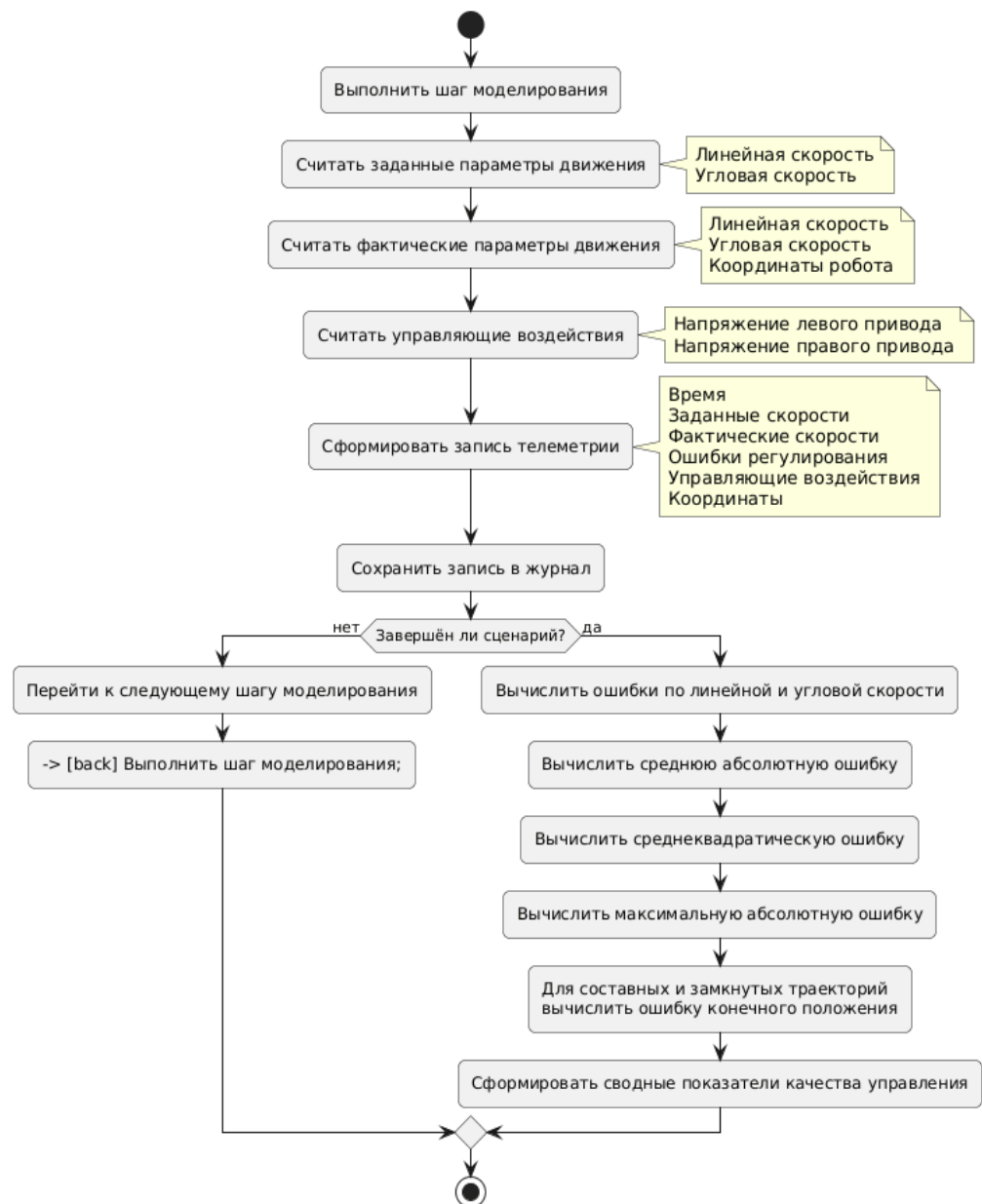


Рисунок 8 — Структура формирования телеметрии и расчёта показателей качества

Как показано на рисунке 8, журнал телеметрии объединяет данные заданного сценария движения, вычислительные результаты рабочего контура и внешние данные среды моделирования. Модуль GPS используется только для

регистрации и последующего анализа результатов испытаний и не участвует в формировании управляющего воздействия.

При интерпретации зарегистрированных данных необходимо учитывать, что фактические линейная и угловая скорости, используемые при оценке скоростных каналов, соответствуют внутреннему состоянию рабочего контура и вычисляются по данным энкодеров ведущих колёс. Поэтому полученные показатели характеризуют воспроизведение заданного скоростного режима по сигналам обратной связи, используемым регуляторами. Данные модуля GPS применяются независимо от скоростного контура только для регистрации координат, построения траектории и вычисления ошибки конечного положения.

Отклонение фактической линейной скорости от значения, заданного сценарием испытания, определяется выражением

$$e_v(k) = v_{\text{зад}}(k) - v_{\text{факт}}(k)$$

Отклонение фактической угловой скорости от значения, заданного сценарием испытания, определяется выражением

$$e_\omega(k) = \omega_{\text{зад}}(k) - \omega_{\text{факт}}(k)$$

где $v_{\text{зад}}(k)$ и $\omega_{\text{зад}}(k)$ — заданные сценарием линейная и угловая скорости платформы на k -м шаге моделирования; $v_{\text{факт}}(k)$ и $\omega_{\text{факт}}(k)$ — зарегистрированные фактические значения линейной и угловой скоростей соответственно.

Значения $e_v(k)$ и $e_\omega(k)$ используются как показатели отклонения фактического движения платформы от заданного сценария. При этом указанные величины не отождествляются с внутренними ошибками регуляторов в переходных процессах, поскольку в компоненте RobotController заданная команда предварительно обрабатывается методом VelocityRamp(...).

Для количественной оценки качества воспроизведения заданного режима движения используются средняя абсолютная ошибка, среднеквадратическая ошибка и максимальная абсолютная ошибка. Указанные показатели вычисляются отдельно для линейного и углового каналов.

Средняя абсолютная ошибка определяется выражением

$$E_{\text{ср.абс}} = \frac{1}{N} \sum_{k=1}^N |e(k)|$$

Среднеквадратическая ошибка определяется выражением

$$E_{\text{ср.кв}} = \sqrt{\frac{1}{N} \sum_{k=1}^N e^2(k)}$$

Максимальная абсолютная ошибка определяется выражением

$$E_{\text{макс}} = \max_{1 \leq k \leq N} |e(k)|$$

В приведённых выражениях N — число зарегистрированных отсчётов телеметрии на рассматриваемом интервале; k — номер отсчёта; $e(k)$ — отклонение по анализируемому каналу. Для линейного канала в качестве данной величины используется $e_v(k)$, а для углового канала — $e_\omega(k)$.

Для сценариев, в которых движение должно завершаться вблизи начальной точки, дополнительно используется ошибка конечного положения. Данная величина определяется как расстояние между начальной и конечной зарегистрированными координатами платформы:

$$E_{\text{кон}} = \sqrt{(x_{\text{кон}} - x_{\text{нач}})^2 + (y_{\text{кон}} - y_{\text{нач}})^2}$$

где $x_{\text{нач}}$ и $y_{\text{нач}}$ — координаты платформы в начале зарегистрированного движения; $x_{\text{кон}}$ и $y_{\text{кон}}$ — координаты платформы после завершения сценария.

Ошибка конечного положения используется как показатель накопленного геометрического отклонения для сценариев, предполагающих завершение движения вблизи начальной точки. Она не является ошибкой позиционного регулирования, поскольку управление положением платформы в глобальной системе координат в разработанной программе не реализуется.

Выбранный набор показателей позволяет оценить различные стороны поведения рабочего контура управления. Средняя абсолютная ошибка характеризует типичный уровень отклонения фактической скорости от заданного

сценарием значения. Среднеквадратическая ошибка учитывает влияние более выраженных отклонений на всём интервале испытания. Максимальная абсолютная ошибка показывает наибольшее зарегистрированное отклонение, возникающее, как правило, в переходных процессах. Ошибка конечного положения отражает накопленный геометрический результат движения в сценариях, предполагающих завершение траектории вблизи начальной точки.

В рамках настоящей работы результаты испытания считаются удовлетворительными, если при выполнении сценария система сохраняет устойчивый характер движения, не демонстрирует нарастающих колебаний, после завершения переходных процессов поддерживает заданный скоростной режим и воспроизводит требуемый характер траектории. Для сценариев, предполагающих завершение движения вблизи начальной точки, дополнительно анализируется ошибка конечного положения с учётом отсутствия отдельного позиционного контура управления.

3.5.2.3. Результаты испытаний

В рамках испытаний рабочего контура управления в среде моделирования рассматриваются три характерных сценария движения мобильной платформы: прямолинейное движение, поворот на месте и движение по квадратной траектории. Выбранная последовательность сценариев позволяет перейти от проверки воспроизведения отдельных заданных скоростей к анализу поведения системы при многократной смене заданного режима движения.

Сценарий прямолинейного движения предназначен для оценки воспроизведения постоянной заданной линейной скорости при нулевом значении угловой скорости. Сценарий поворота на месте используется для оценки воспроизведения заданной угловой скорости при отсутствии требуемого поступательного движения. В данных сценариях основное внимание уделяется отклонениям по соответствующему анализируемому каналу.

Сценарий движения по квадратной траектории содержит чередование прямолинейных участков и поворотов на месте. Он используется для оценки

поведения системы в условиях повторяющихся переходных процессов и многократной смены заданного режима движения. Поскольку заданный маршрут предполагает завершение движения вблизи начальной точки, для него также анализируется ошибка конечного положения.

3.5.2.3.1. Прямолинейное движение

В данном сценарии задаётся постоянная линейная скорость платформы при нулевом значении заданной угловой скорости:

$$v_{зад} = 0,2 \text{ м/с}$$

$$\omega_{зад} = 0 \text{ рад/с}$$

Данный сценарий предназначен для оценки воспроизведения заданного поступательного режима движения. Поскольку в составе команды ненулевое значение имеет только линейная скорость, анализ результатов выполняется по зарегистрированной фактической линейной скорости платформы и её отклонению от заданного сценарием значения. На рисунке 9 представлены зависимости заданной и фактической линейных скоростей платформы от времени.

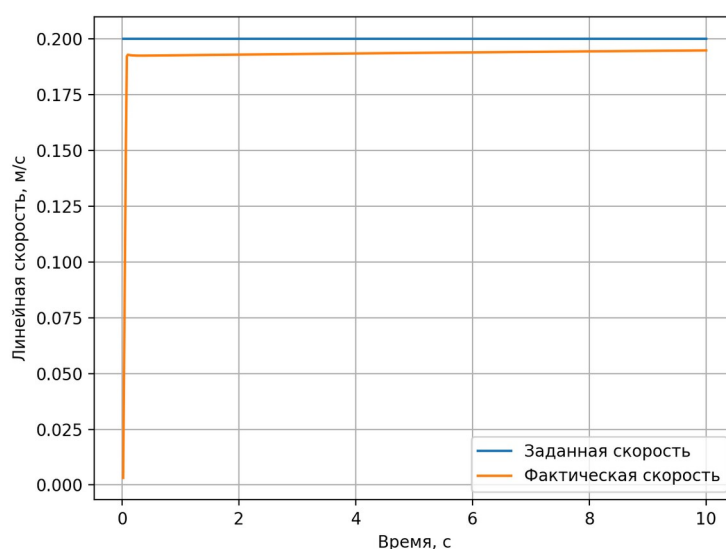


Рисунок 9 — Заданные и фактические значения линейной скорости при прямолинейном движении

Как показано на рисунке 9, после начала сценария фактическая линейная скорость стремится к заданному значению в ходе переходного процесса,

обусловленного разгоном платформы из состояния покоя. На рисунке 10 представлено отклонение фактической линейной скорости от значения, заданного сценарием движения.

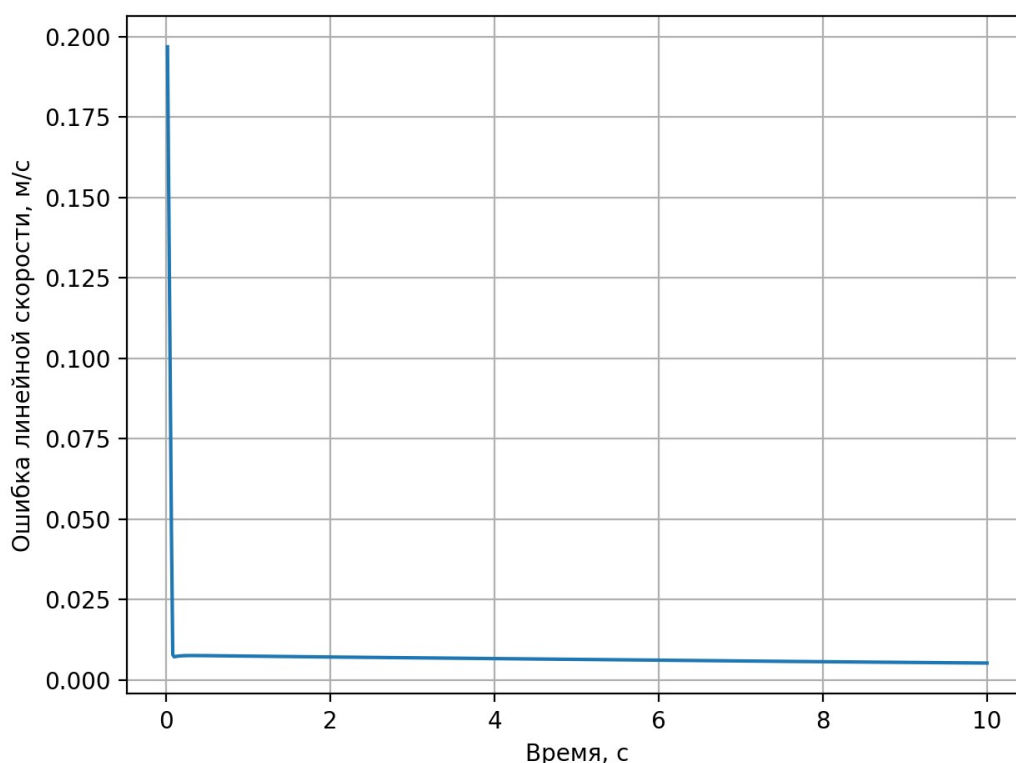


Рисунок 10 — Отклонение линейной скорости от заданного значения при прямолинейном движении

Наибольшее отклонение линейной скорости наблюдается в начальной части эксперимента, когда фактическая скорость платформы ещё не достигла заданного значения. По мере завершения переходного процесса отклонение уменьшается и далее остаётся ограниченным.

Результаты количественной оценки линейного канала имеют вид:

$$E_{\text{ср.абс},v}=0,007091 \text{ м/с}$$

$$E_{\text{макс},v}=0,196803 \text{ м/с}$$

$$E_{\text{ср.кв},v}=0,012497 \text{ м/с}$$

Полученные результаты показывают, что максимальное отклонение связано с начальным переходным процессом. После завершения разгона рабочий контур обеспечивает поддержание заданной линейной скорости без нарастания

отклонения. Следовательно, в рассматриваемом сценарии обеспечивается устойчивое воспроизведение заданного прямолинейного режима движения.

3.5.2.3.2. Поворот на месте

В данном сценарии задаётся постоянная угловая скорость платформы при нулевом значении заданной линейной скорости:

$$v_{\text{зад}} = 0 \text{ м/с}$$

$$\omega_{\text{зад}} = 1,0 \text{ рад/с}$$

Данный сценарий предназначен для оценки воспроизведения заданного вращательного режима движения. Поскольку в составе команды ненулевое значение имеет только угловая скорость, анализ результатов выполняется по зарегистрированной фактической угловой скорости платформы и её отклонению от заданного сценарием значения. На рисунке 11 представлены зависимости заданной и фактической угловых скоростей платформы от времени.

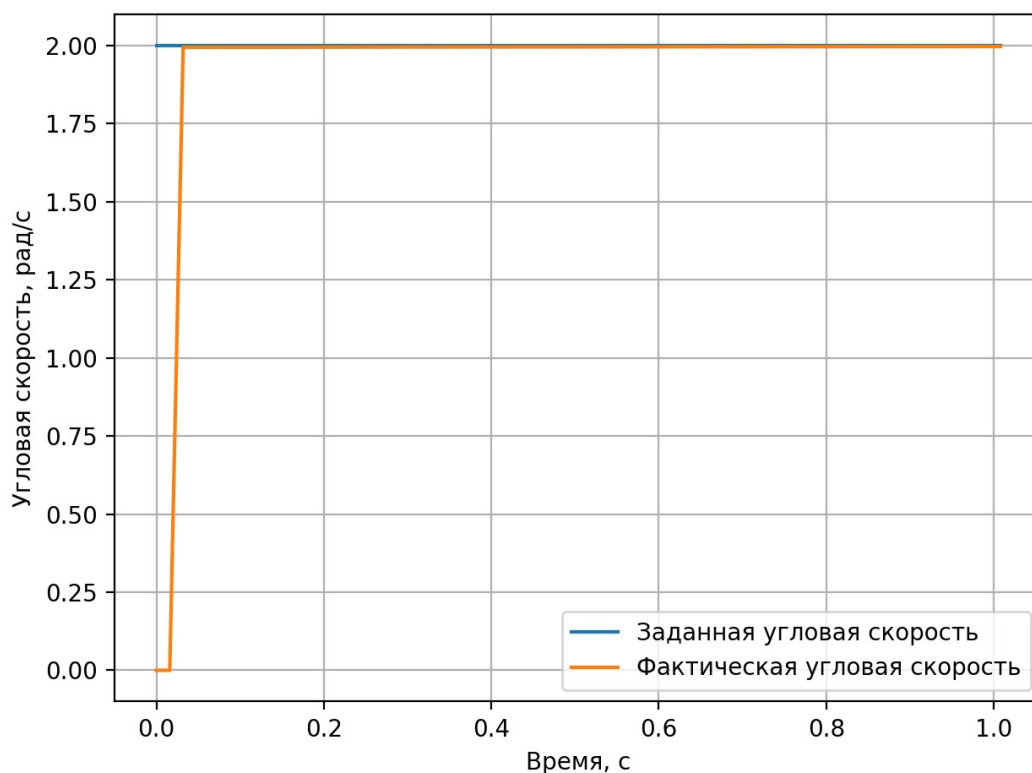


Рисунок 11 — Заданные и фактические значения угловой скорости при повороте на месте

Как показано на рисунке 11, после начала сценария фактическая угловая скорость достигает заданного значения в ходе переходного процесса. После его завершения наблюдается стабилизация угловой скорости вблизи заданного уровня. На рисунке 12 представлено отклонение фактической угловой скорости от значения, заданного сценарием движения.

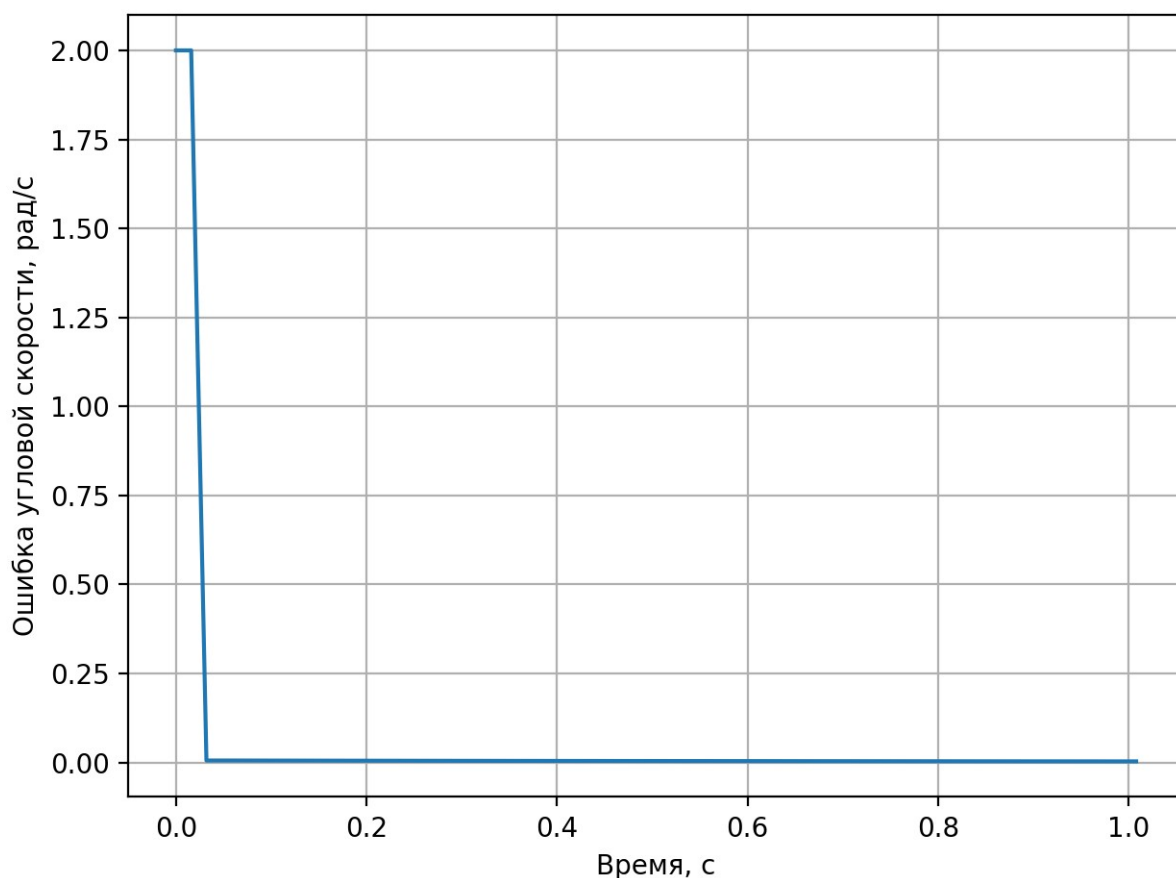


Рисунок 12 — Отклонение угловой скорости от заданного значения
при повороте на месте

Наибольшее отклонение угловой скорости наблюдается в начальной части эксперимента, когда фактическая скорость вращения платформы ещё не достигла заданного значения. По мере завершения переходного процесса отклонение уменьшается и далее остаётся ограниченным.

Результаты количественной оценки углового канала имеют вид:

$$E_{\text{ср.абс},\omega} = 0,018865 \text{ рад/с}$$

$$E_{\text{ср.кв},\omega} = 0,069911 \text{ рад/с}$$

$$E_{\text{макс}, \omega} = 1,0 \text{ рад/с}$$

Полученные результаты показывают, что максимальное отклонение связано с начальным переходным процессом при переходе от состояния покоя к заданному режиму вращения. После завершения переходного процесса рабочий контур обеспечивает поддержание заданной угловой скорости без нарастания отклонения. Следовательно, в рассматриваемом сценарии обеспечивается устойчивое воспроизведение заданного режима поворота платформы на месте.

3.5.2.3.3. Движение по квадратной траектории

Сценарий движения по квадратной траектории используется для оценки работы рабочего контура управления в условиях многократной смены заданного режима движения. В ходе сценария участки прямолинейного движения последовательно чередуются с поворотами платформы, соответствующими изменению направления движения приблизительно на угол $\frac{\pi}{2}$. Такое построение маршрута приводит к возникновению повторяющихся переходных процессов в линейном и угловом каналах.

В отличие от ранее рассмотренных сценариев, движение по квадратной траектории позволяет оценить поведение рабочего контура при последовательном выполнении различных режимов движения в рамках одного маршрута. Поскольку заданная траектория предполагает завершение движения вблизи начальной точки, для данного сценария наряду с отклонениями линейной и угловой скоростей анализируется ошибка конечного положения. На рисунке 13 представлена зарегистрированная траектория движения мобильной платформы.

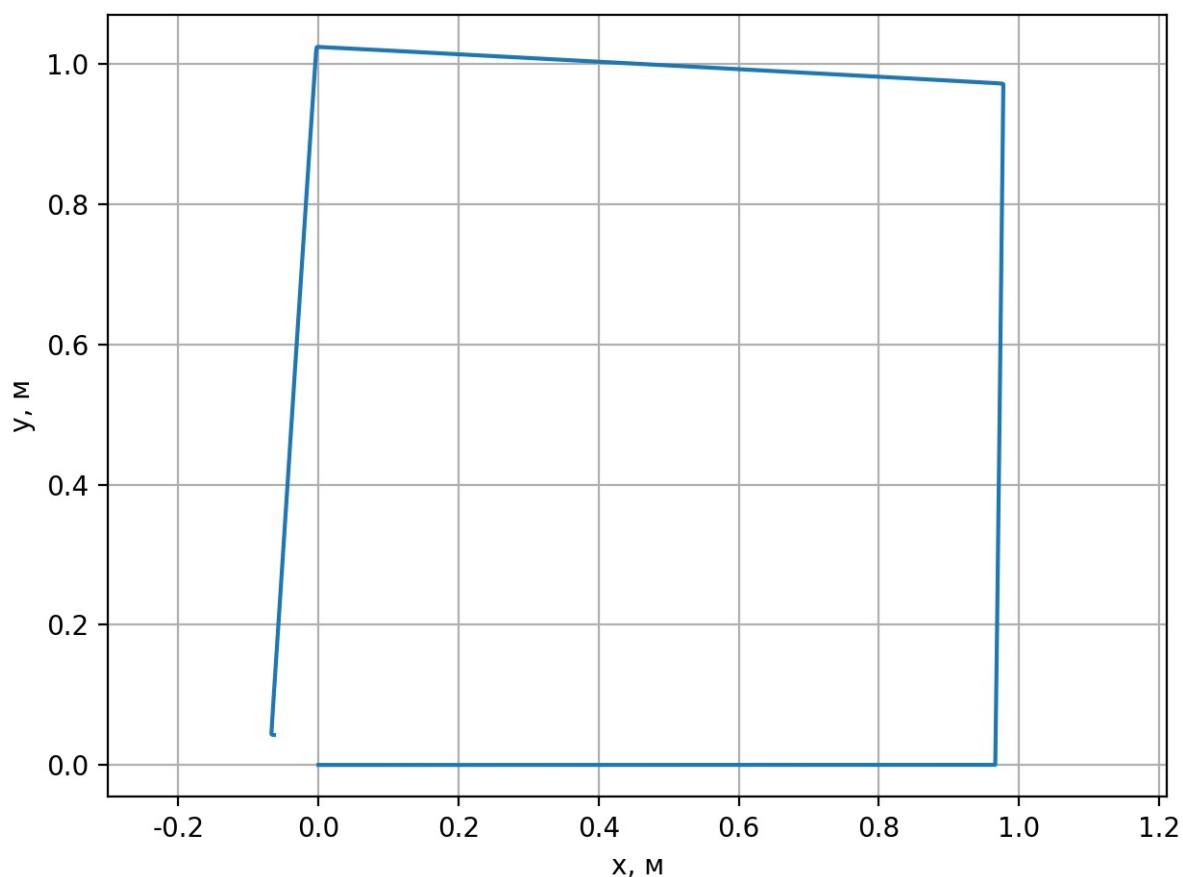


Рисунок 13 — Траектория движения мобильной платформы при движении по квадратной траектории

Как показано на рисунке 13, фактическая траектория в целом воспроизводит форму заданного квадратного маршрута. После выполнения последовательности прямолинейных участков и поворотов конечное положение платформы располагается вблизи начальной точки, однако полного совпадения положений не наблюдается, что свидетельствует о накоплении геометрического отклонения на протяжении составного замкнутого маршрута. На рисунке 14 представлено отклонение фактической линейной скорости от значения, заданного сценарием движения.

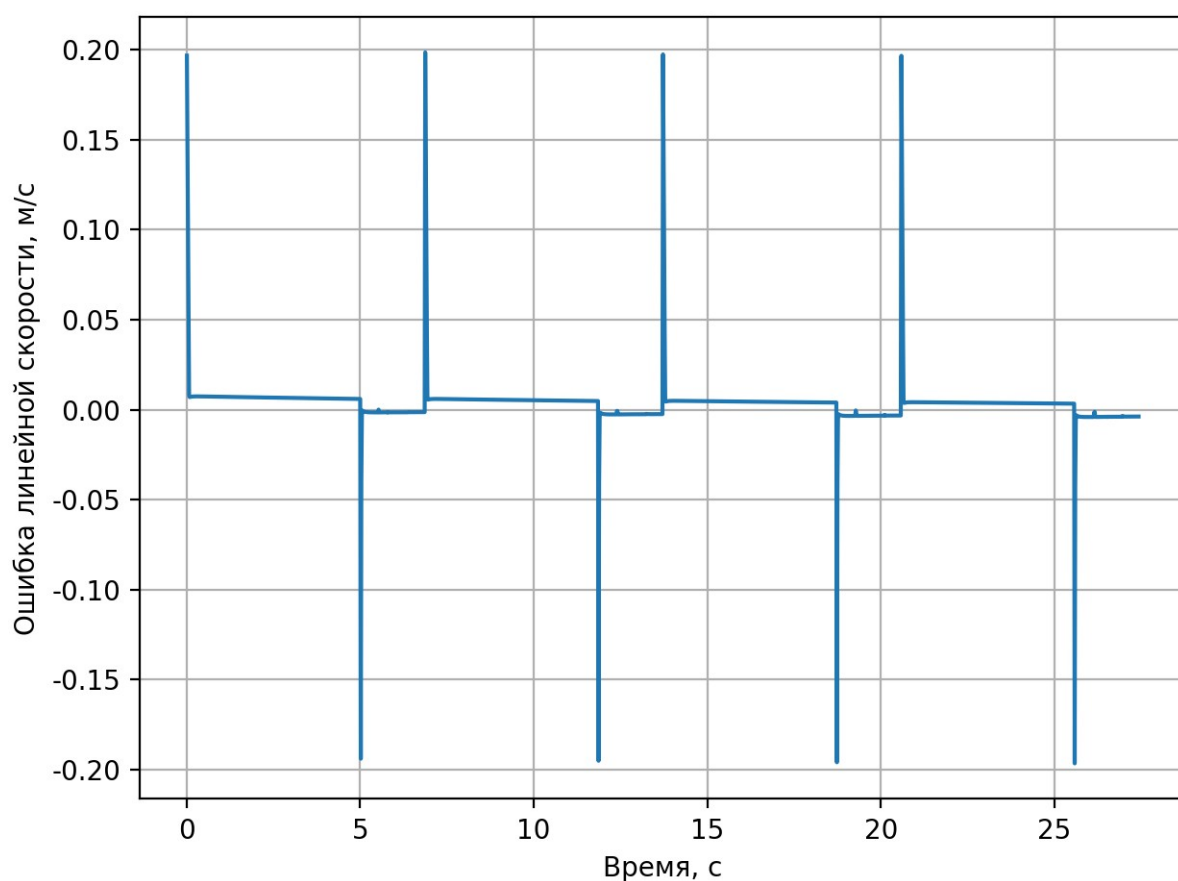


Рисунок 14 — Отклонение линейной скорости от заданного значения при движении по квадратной траектории

Как показано на рисунке 14, увеличение отклонения линейной скорости наблюдается преимущественно в моменты начала прямолинейных участков и перехода к режиму поворота. После завершения каждого переходного процесса отклонение уменьшается, что указывает на восстановление требуемого поступательного режима движения на соответствующих участках маршрута. На рисунке 15 представлено отклонение фактической угловой скорости от значения, заданного сценарием движения.

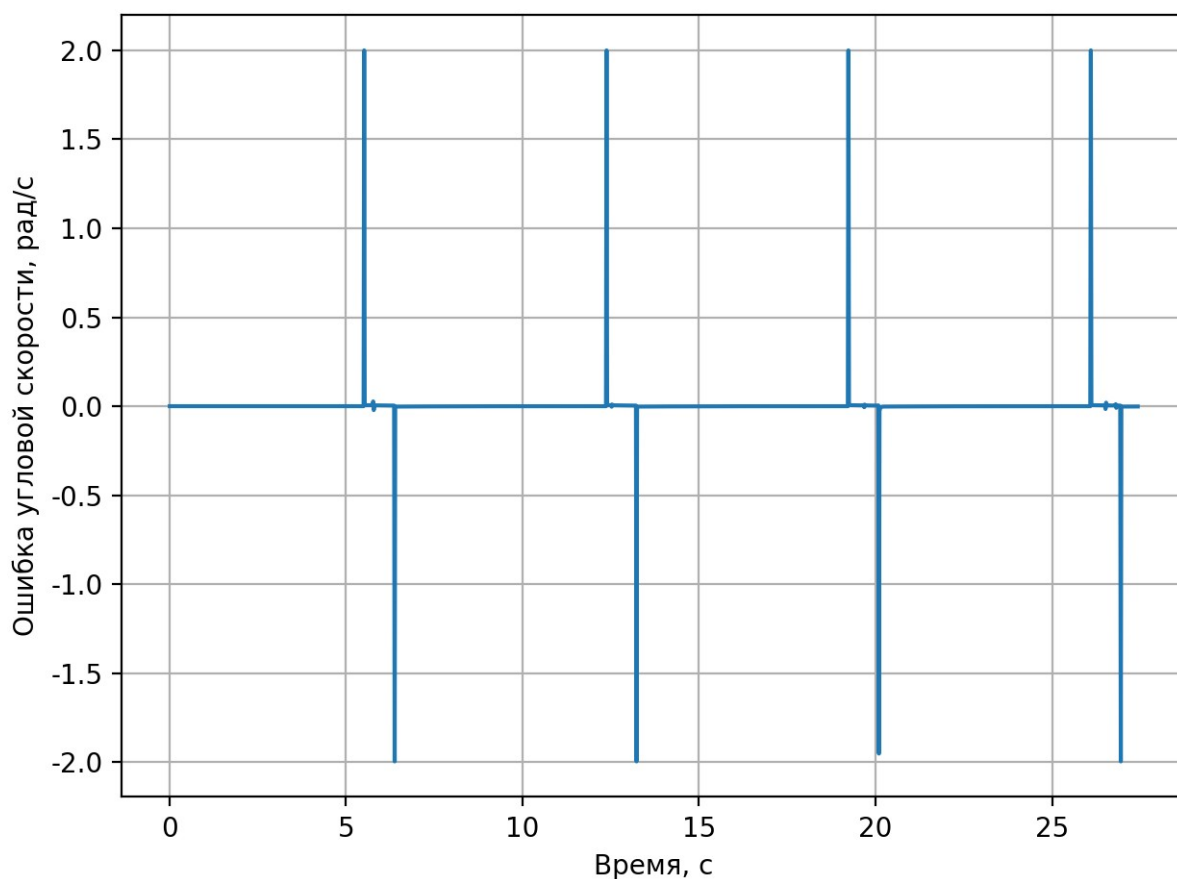


Рисунок 15 — Отклонение угловой скорости от заданного значения при движении по квадратной траектории

Как показано на рисунке 15, отклонения угловой скорости возникают при выполнении последовательных поворотов платформы. Их появление связано с переходом от прямолинейного режима к вращательному и обратно. При этом на графике не наблюдается нарастающего характера отклонений от одного поворота к следующему, что свидетельствует об устойчивом поведении углового канала при повторяющейся смене режима движения.

Результаты количественной оценки данного сценария имеют вид:

$$E_{\text{ср.абс},v}=0,006227 \text{ м/с}$$

$$E_{\text{ср.абс},\omega}=0,048406 \text{ рад/с}$$

$$E_{\text{кон}}=0,156 \text{ м}$$

Полученные результаты показывают, что при многократном чередовании прямолинейного движения и поворотов рабочий контур сохраняет ограниченный

уровень отклонений линейной и угловой скоростей от заданных значений. Зарегистрированная ошибка конечного положения указывает на наличие накопленного геометрического отклонения, однако фактическая траектория сохраняет общую форму заданного замкнутого маршрута.

Таким образом, сценарий движения по квадратной траектории подтверждает работоспособность рабочего контура управления в условиях повторяющихся переходных процессов и многократной смены заданного режима движения. При этом результат согласуется с ограничениями разработанной программы: управление скоростями обеспечивает воспроизведение характера маршрута, но не устраняет полностью накопленное отклонение конечного положения при отсутствии отдельного позиционного контура.

3.5.2.4. Выводы по испытаниям в среде моделирования

Для сопоставления результатов проведённых испытаний сводные показатели качества управления представлены в таблице 3.

Таблица 3 — Сводные результаты испытаний рабочего контура управления в среде моделирования

Сценарий	$E_{\text{ср.абс}, v}$ м/с	$E_{\text{ср.абс}, \omega}$ рад/с	$E_{\text{макс}, v}$ м/с	$E_{\text{макс}, \omega}$ рад/с	$E_{\text{ср.кв}, v}$ м/с	$E_{\text{ср.кв}, \omega}$ рад/с	$E_{\text{кон}}$ м
Прямолинейное движение	0,007091	—	0,1968 03	—	0,0124 97	—	—
Поворот на месте	—	0,018865	—	1,0	—	0,069911	—
Движение по квадратной траектории	0,006227	0,048406	—	—	—	—	0,156

Сводные результаты показывают, что в сценариях прямолинейного движения и поворота на месте наибольшие зарегистрированные отклонения возникают в начальных переходных процессах. После их завершения фактические значения соответствующей скорости стабилизируются вблизи заданного сценарием уровня. Тем самым данные сценарии подтверждают

воспроизведение рабочим контуром отдельно заданных поступательного и вращательного режимов движения.

Сценарий движения по квадратной траектории позволяет оценить работу контура в условиях многократного чередования прямолинейного движения и поворотов. Результаты данного испытания показывают, что при повторяющихся переходных процессах фактическая траектория сохраняет общую форму заданного маршрута, а накопленное отклонение конечного положения остаётся ограниченным.

Таким образом, проведённые испытания в среде моделирования подтверждают работоспособность разработанного рабочего контура управления при воспроизведении рассмотренных скоростных режимов движения. В представленных сценариях не наблюдается нарастающего характера отклонений, препятствующего выполнению заданного движения. При этом испытания на замкнутом маршруте выявляют ограничение принятой структуры управления: без отдельного позиционного контура накопленные отклонения скоростей могут приводить к несовпадению конечного и начального положений платформы.

Полученные результаты характеризуют качество работы именно скоростного контура управления и не должны интерпретироваться как подтверждение реализации управления положением мобильной платформы в глобальной системе координат.

3.6. Выводы

В данной главе определены назначение и функциональные требования к программе управления ходовой частью мобильной платформы. В качестве основного предмета разработки принят рабочий контур скоростного управления, обеспечивающий приём заданных линейной и угловой скоростей, получение обратной связи по данным энкодеров ведущих колёс, вычисление управляющих воздействий и передачу рассчитанных значений приводам.

Разработана архитектура программы, основанная на разделении организации управления, реализации закона управления и средств

взаимодействия с измерительной и исполнительной частью. Центральными компонентами рабочего контура являются `RobotController` и `Robot`. Модельная составляющая управляющего воздействия формируется компонентом `FFModel`, корректирующая составляющая — двумя регуляторами на основе класса `Math::PID`, настроенными в рассматриваемой конфигурации по ПИ-закону. Взаимодействие с энкодерами и приводами отделено от вычислительной логики посредством интерфейсов аппаратной абстракции.

На уровне программной реализации рассмотрен дискретный цикл управления, включающий обновление измерительных данных, формирование текущего скоростного состояния платформы, вычисление управляющего воздействия и его передачу исполнительной части. Показано, что используемая в регулировании обратная связь формируется по линейным скоростям ведущих колёс, а результирующее воздействие представляет собой объединение модельной и корректирующей составляющих. Кроме того, в рабочем контуре предусмотрена обработка недопустимых состояний, при которых штатное вычисление либо применение нового воздействия прекращается.

Для проверки программной реализации выполнены модульные испытания компонентов рабочего контура и испытания в среде моделирования `Webots`. Модульная проверка охватывает получение энкодерной обратной связи, формирование модельной и корректирующей составляющих, получение результирующего воздействия и его передачу исполнительной части.

В ходе испытаний в среде моделирования рассмотрены три режима движения: прямолинейное движение, поворот на месте и движение по квадратной траектории. Результаты показали, что разработанный рабочий контур обеспечивает воспроизведение заданных линейной и угловой скоростей, а также сохраняет работоспособность при многократной смене режима движения. Основные зарегистрированные отклонения связаны с переходными процессами при начале движения и смене задаваемого режима.

Испытания на замкнутом маршруте показало наличие накопленного отклонения конечного положения. Данный результат соответствует принятой постановке задачи: разработанная программа реализует управление линейной и угловой скоростями платформы, но не включает контур коррекции её положения в глобальной системе координат.

Таким образом, в главе разработан и проверен рабочий программный контур управления ходовой частью мобильной платформы, соответствующий принятому закону управления и архитектурному разделению компонентов. Полученные результаты создают основание для последующей аппаратно-программной проверки выполнения вычислительной части управления на микроконтроллерной платформе STM32.

4. Встраиваемая реализация и аппаратно-программная проверка программы управления на платформе STM32

4.1. Организация аппаратно-программного контура проверки

Разработанная программа управления ходовой частью четырёхколёсного мобильного робота предназначена для выполнения на микроконтроллерной платформе STM32. Поэтому проверка программы должна подтверждать не только корректность закона управления при работе с моделью объекта, но и выполнение разработанной вычислительной логики непосредственно на микроконтроллере [6, 19, 20].

В разделе 3.5.2 была рассмотрена программная проверка разработанной системы управления в среде Webots [15, 16]. Для подтверждения выполнения программы на целевой микроконтроллерной платформе дополнительно реализован аппаратно-программный контур с использованием STM32.

В данном контуре среда Webots сохраняет функцию модели мобильной платформы, а вычислительная часть программы управления выполняется в составе прошивки STM32. На каждом шаге моделирования сведения о состоянии модели и команда движения передаются на микроконтроллер через последовательный канал связи. Микроконтроллер рассчитывает управляющие воздействия и возвращает их в Webots для применения к двигателям модели [18]. Общий вид использованного стенда представлен на рисунке 16.

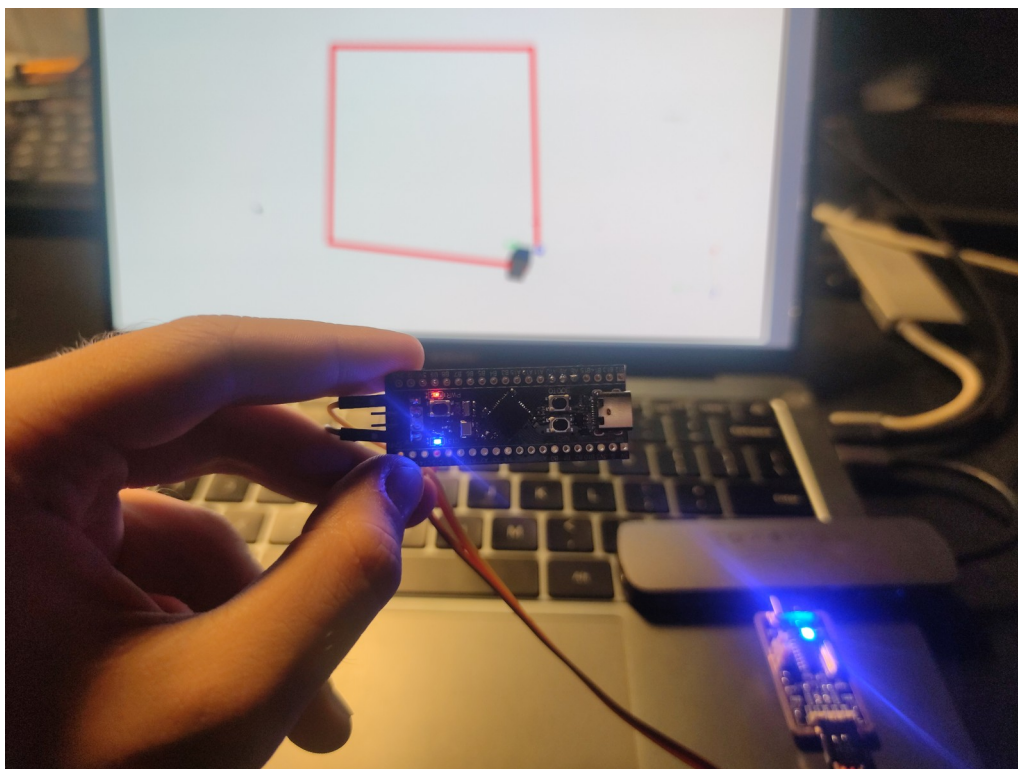


Рисунок 16 — Общий вид аппаратно-программного стенда для проверки программы управления

Структура реализованного контура представлена на рисунке 17.

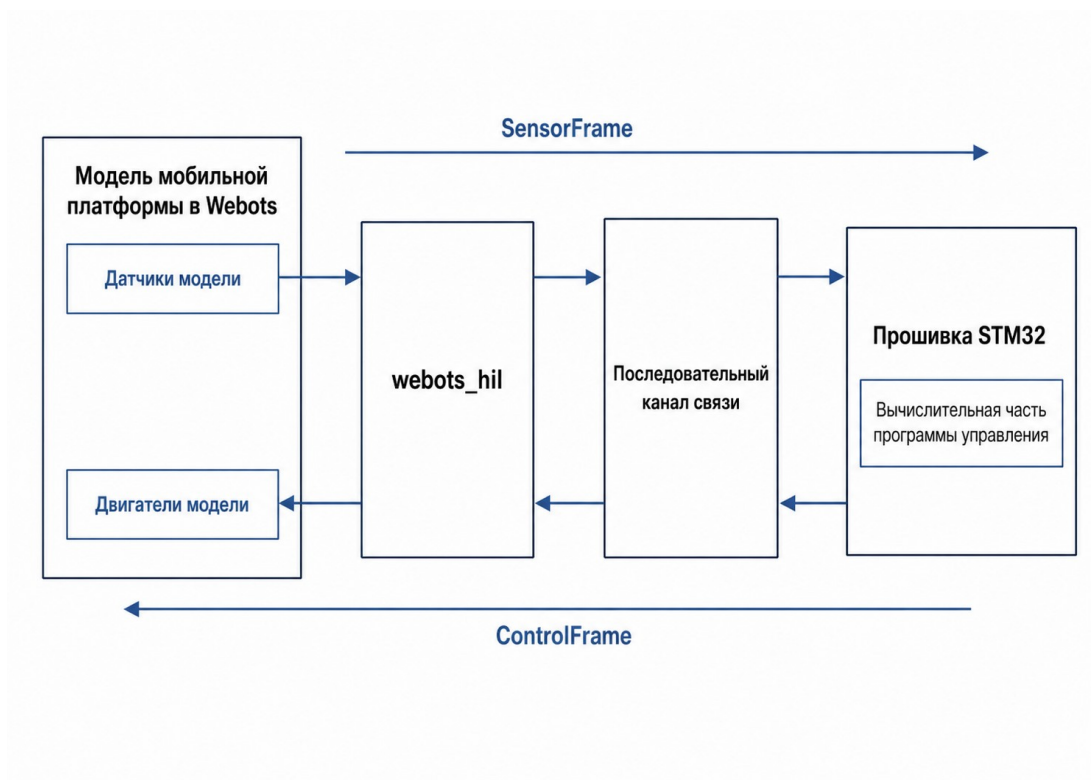


Рисунок 17 — Структура аппаратно-программного контура проверки

В проекте для данного режима реализован отдельный управляющий модуль `webots_hil`, подключённый в составе подсистемы моделирования наряду с модулем `webots_sil`. В отличие от полностью программной проверки, в рассматриваемом режиме вычислительная часть программы управления выполняется не на компьютере, а в составе прошивки STM32.

Изготовление физической мобильной платформы и проверка её механической части в объём настоящей работы не входят. В эксперименте проверяется программа управления, выполняемая на STM32, при взаимодействии с моделируемым объектом управления.

4.2. Реализация прошивки и обмена данными

Встраиваемая программа формируется в виде исполняемого образа `firmware.elf`. В состав прошивки включаются библиотеки `robot` и `robotmath`, содержащие вычислительную логику системы управления, а также модули `robot_hil`, `robot_hil_stm32` и `stm32_platform`, обеспечивающие обмен с моделируемой средой и выполнение программы на STM32. Кросс-компиляция для целевой платформы организована с использованием файла конфигурации CMake [14]. Дополнительно при сборке формируются файлы `firmware.bin` и `firmware.hex`, предназначенные для загрузки в микроконтроллер.

Основные компоненты аппаратно-программной проверки приведены в таблице 4.

Таблица 4 — Компоненты аппаратно-программной проверки

Компонент	Назначение
<code>robot, robotmath</code>	вычислительная логика программы управления
<code>robot_hil</code>	формат обменных кадров и общие средства взаимодействия
<code>robot_hil_stm32</code>	передача кадров через последовательный интерфейс STM32
<code>firmware.elf</code>	исполняемый образ программы на микроконтроллере

Компонент	Назначение
webots_hil	передача состояния модели на STM32 и применение рассчитанных воздействий
STM32F411CEUx.ld	размещение кода и данных прошивки в памяти микроконтроллера

Обмен между Webots и STM32 выполняется кадрами фиксированного формата. Для передачи входных данных используется кадр SensorFrame, содержащий сведения о текущем состоянии модели и задании на движение. Результат вычисления возвращается в кадре ControlFrame. В рассматриваемой реализации управляющее воздействие представлено рассчитанными значениями напряжения для левого и правого двигателя. Содержание обменных кадров приведено в таблице 5.

Таблица 5 — Содержание обменных кадров

Кадр	Передаваемые данные	Назначение
SensorFrame	номер кадра, шаг дискретизации, значения энкодеров, данные инерциального датчика, заданные линейная и угловая скорости	передача входных данных для расчёта управления
ControlFrame	номер кадра, напряжения левого и правого двигателей, текущие линейная и угловая скорости, состояние обработки	передача результата управляющего шага

Размер кадра SensorFrame составляет 48 байт, размер кадра ControlFrame — 24 байта. Для контроля обмена используются служебные признаки кадров, версия формата, номер кадра seq и контрольная сумма CRC-16. Номер кадра позволяет сопоставить результат расчёта с соответствующим шагом моделирования, а контрольная сумма используется для выявления нарушения целостности передаваемых данных.

Последовательный обмен на стороне микроконтроллера реализуется с использованием периферии UART и программных средств STM32 HAL [19, 20, 21]. Сопряжение обмена с вычислительной частью прошивки показано на рисунке 18.

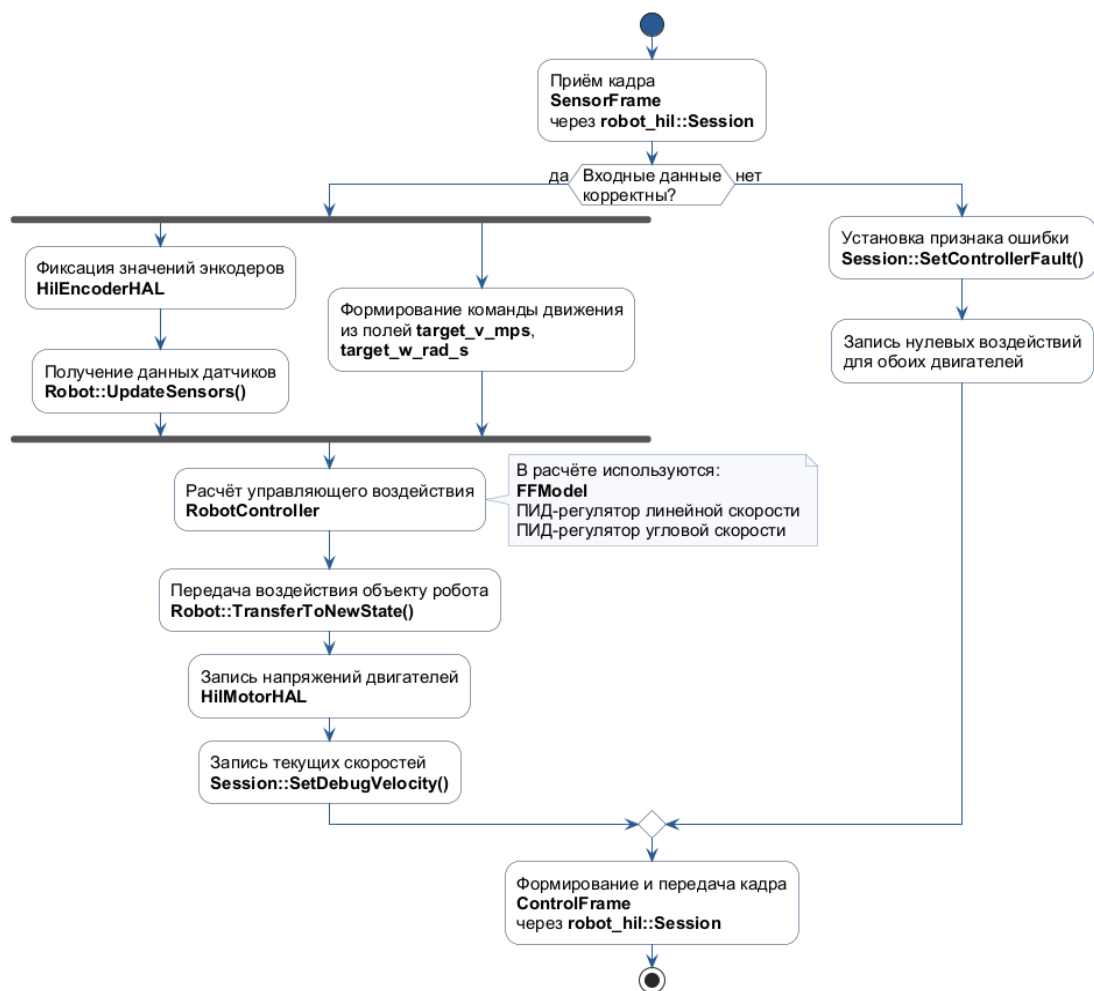


Рисунок 18 — Выполнение управляющего шага в прошивке STM32

В прошивке создаются объект обмена `robot_hil::Session`, реализации аппаратной абстракции `HilEncoderHAL`, `HilImuHAL` и `HilMotorHAL`, а также компоненты системы управления `Robot`, `FFModel`, два ПИД-регулятора и `RobotController`. Классы `HilEncoderHAL` и `HilImuHAL` предоставляют вычислительной части данные, полученные во входном кадре, а `HilMotorHAL` записывает рассчитанные напряжения двигателей в выходной кадр.

Один шаг работы прошивки включает приём кадра `SensorFrame`, проверку входных данных, обновление состояния мобильной платформы, расчёт управляющего воздействия объектом `RobotController`, формирование кадра `ControlFrame` и передачу результата обратно в `Webots`. При некорректных входных данных в выходной кадр записываются нулевые значения воздействий и признак ошибки.

Таким образом, в аппаратно-программном контуре STM32 выполняет не только операции передачи данных, но и основную вычислительную цепочку разработанной программы управления. Рассчитанные микроконтроллером напряжения передаются в Webots и используются для формирования движения модели мобильной платформы.

4.3. Методика проверки

Для экспериментальной проверки использован сценарий движения по квадратной траектории. Такой сценарий включает последовательное выполнение прямолинейных участков и поворотов, поэтому требует многократного согласованного обмена между Webots и STM32 и расчёта различных управляющих воздействий при смене режима движения.

В модуле webots_hil используются параметры модели, согласованные с прошивкой: радиус колеса 0,025 м, ширина колеи 0,100 м, число импульсов энкодера на оборот 2048 и предельное напряжение двигателя 12 В. Обмен с микроконтроллером выполняется через последовательный интерфейс со скоростью 115200 бит/с.

Сценарий состоит из четырёх последовательных повторений набора команд, приведённого в таблице 6. Каждое повторение формирует одну сторону квадратной траектории и последующий поворот платформы.

Таблица 6 — Набор команд одного участка сценария движения по квадратной траектории

Режим	Линейная скорость, м/с	Угловая скорость, рад/с	Длительность, с
Прямолинейное движение	0,20	0,00	5,000
Остановка	0,00	0,00	0,500
Поворот на месте	0,00	2,00	0,784
Остановка	0,00	0,00	0,500

При выполнении сценария webots_hil формирует входные кадры, передаёт их на STM32, принимает выходные кадры и применяет содержащиеся в них

напряжения к двигателям модели. Результаты записываются в файл `square_hil.csv`, содержащий заданные и измеренные скорости, ошибки регулирования, напряжения двигателей, координаты модели, признак совпадения номера кадров и состояние обработки.

Для оценки качества движения используются ошибки линейной и угловой скорости:

$$\begin{aligned} e_v(t) &= v_{зад}(t) - v(t) \\ e_\omega(t) &= \omega_{зад}(t) - \omega(t) \end{aligned}$$

Среднеквадратические ошибки определяются выражениями:

$$\begin{aligned} e_{v,RMS} &= \sqrt{\frac{1}{N} \sum_{i=1}^N e_v^2(i)} \\ e_{\omega,RMS} &= \sqrt{\frac{1}{N} \sum_{i=1}^N e_\omega^2(i)} \end{aligned}$$

Корректность взаимодействия с STM32 оценивается по признаку `seq_ok`, показывающему соответствие входного и выходного кадров, и по полю `status`, содержащему состояние обработки на стороне прошивки.

4.4. Результаты аппаратно-программной проверки

В ходе аппаратно-программной проверки зарегистрировано 1653 шага обмена общей длительностью 26,528 с. Во всех записанных шагах номера входного и выходного кадров совпали, а ненулевые состояния ошибки со стороны прошивки отсутствуют. Траектория движения модели представлена на рисунке 19.

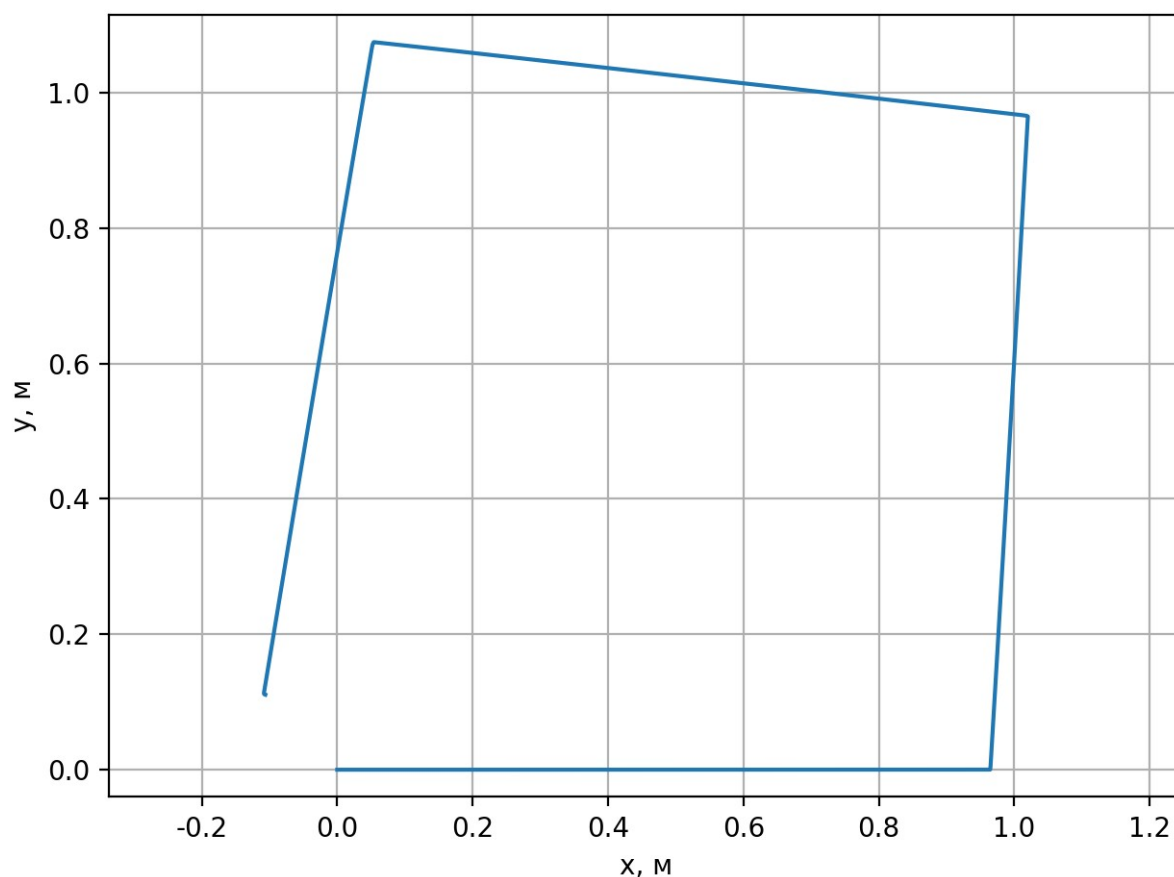


Рисунок 19 — Траектория движения мобильной платформы
при аппаратно-программной проверке

На рисунке 19 представлены участки движения модели, сформированные воздействиями, рассчитанными в прошивке STM32. Полученная траектория подтверждает функционирование замкнутого аппаратно-программного контура при последовательной смене режимов прямолинейного движения и поворота на месте. Заданная и измеренная линейная скорости мобильной платформы представлены на рисунке 20.

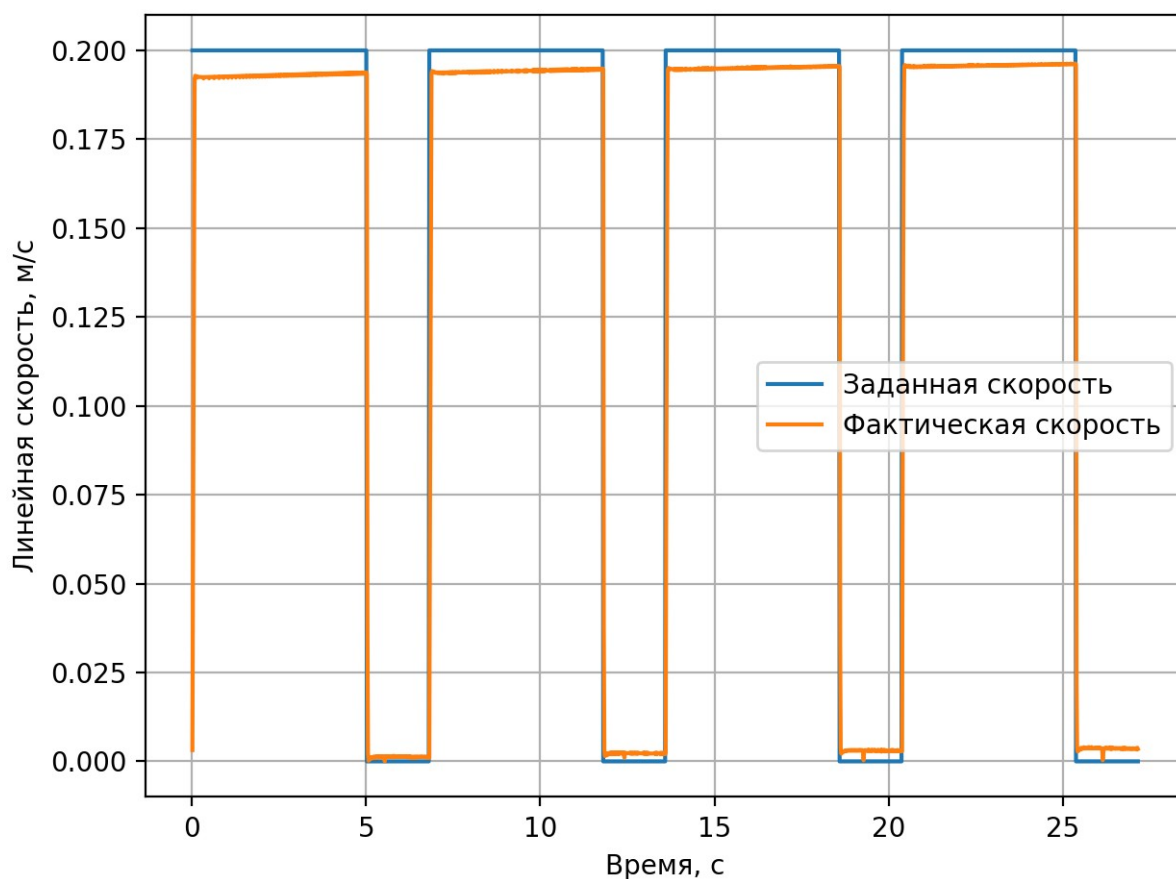


Рисунок 20 — Заданная и измеренная линейная скорости при аппаратно-программной проверке

На рисунке 20 видно, что на участках прямолинейного движения измеренная линейная скорость воспроизводит заданное значение. При переходе к движению и остановке возникают кратковременные отклонения, обусловленные переходным процессом в замкнутом контуре управления. После завершения переходного процесса измеренная скорость приближается к заданной. Заданная и измеренная угловая скорости мобильной платформы представлены на рисунке 21.

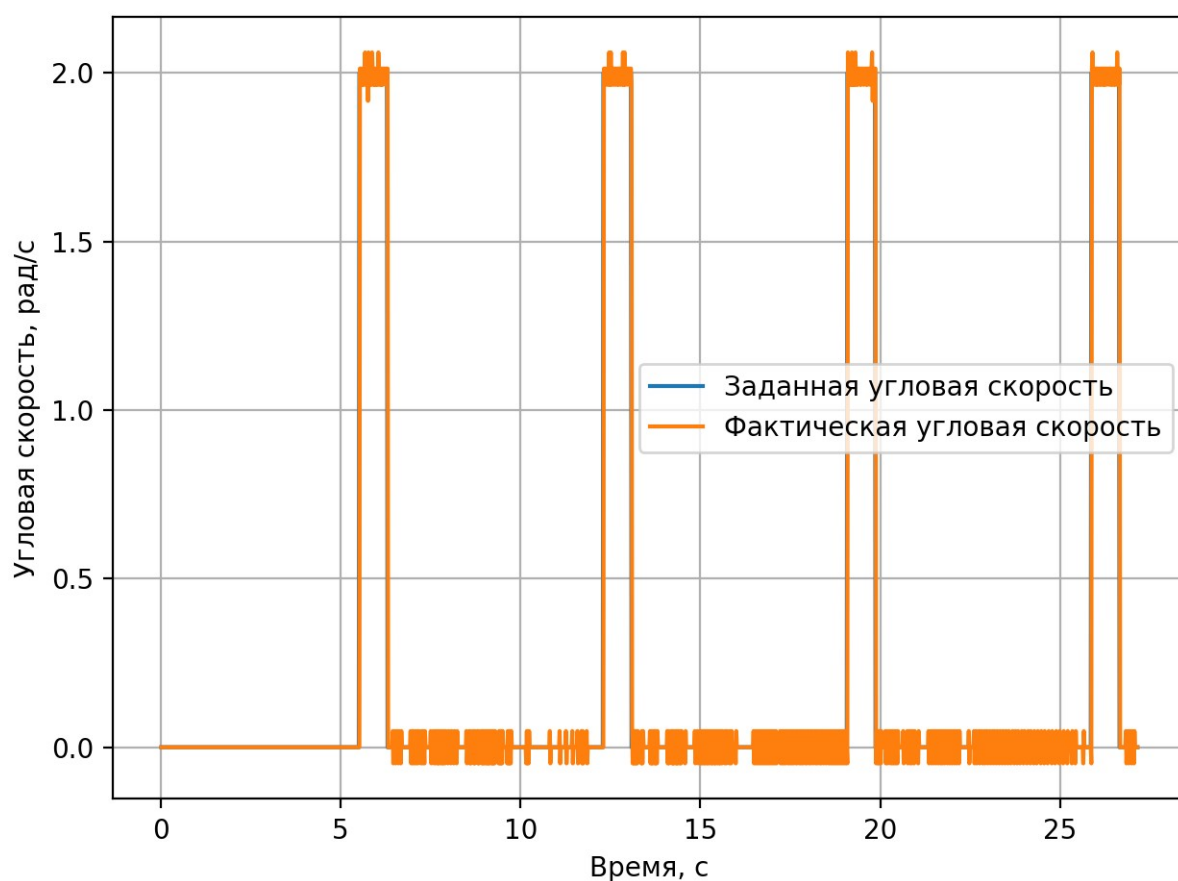


Рисунок 21 — Заданная и измеренная угловая скорости
при аппаратно-программной проверке

На рисунке 21 представлены участки поворота платформы на месте. При подаче команды на поворот измеренная угловая скорость изменяется в соответствии с заданным режимом движения. Наибольшие отклонения возникают в моменты начала и завершения поворота, когда заданная угловая скорость изменяется скачкообразно. Ошибка регулирования линейной скорости представлена на рисунке 22.

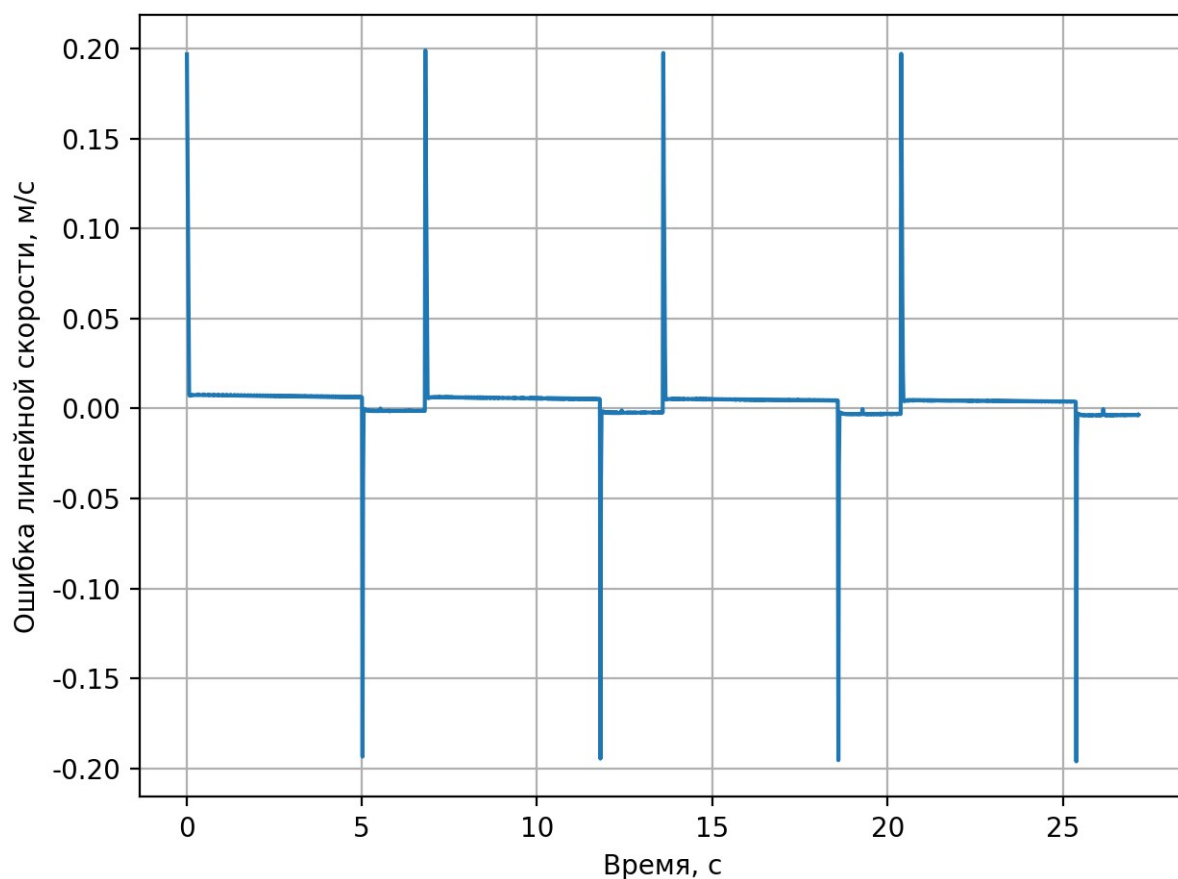


Рисунок 22 — Ошибка регулирования линейной скорости
при аппаратно-программной проверке

Из рисунка 22 следует, что наибольшие значения ошибки линейной скорости соответствуют переходам между остановкой и прямолинейным движением. На установившихся участках прямолинейного движения ошибка уменьшается и сохраняется ограниченной. Данный результат характеризует работу регулятора линейной скорости в составе программы, выполняемой на STM32. Ошибка регулирования угловой скорости представлена на рисунке 23.

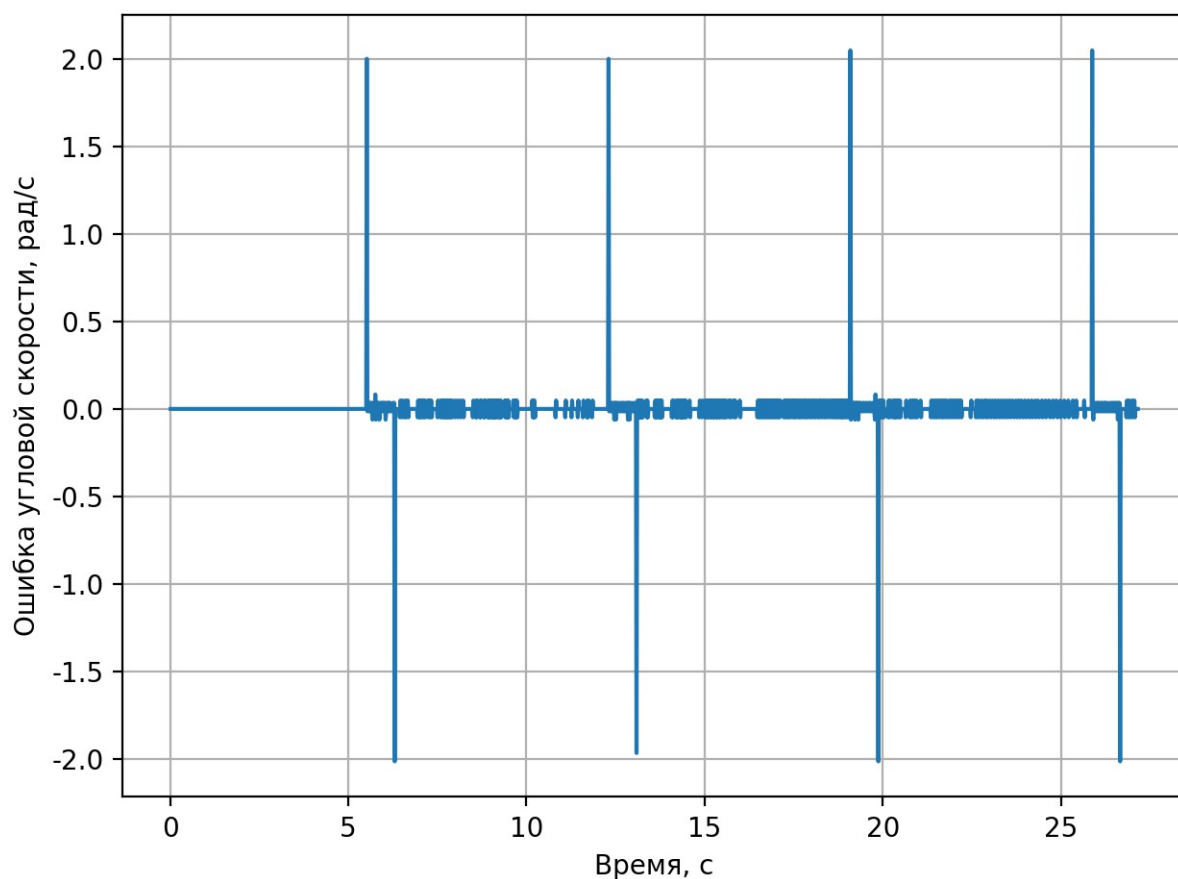


Рисунок 23 — Ошибка регулирования угловой скорости
при аппаратно-программной проверке

На рисунке 23 видно, что основные отклонения угловой скорости возникают в переходных режимах, связанных с началом и окончанием поворота на месте. При выполнении заданного поворота ошибка уменьшается после начального переходного процесса. Такой характер ошибки соответствует используемому сценарию, в котором команда угловой скорости изменяется ступенчато. Количественные результаты проверки приведены в таблице 7.

Таблица 7 — Результаты аппаратно-программной проверки

Показатель	Значение
Число зарегистрированных шагов обмена	1653
Продолжительность зарегистрированного интервала, с	26,528
Доля кадров с совпадающим номером, %	100
Число кадров с состоянием ошибки	0
Среднее абсолютное значение ошибки линейной скорости, м/с	0,00528
Среднеквадратическая ошибка линейной скорости, м/с	0,01884
Максимальная абсолютная ошибка линейной скорости, м/с	0,19810
Среднее абсолютное значение ошибки угловой скорости, рад/с	0,07467
Среднеквадратическая ошибка угловой скорости, рад/с	0,22209
Максимальная абсолютная ошибка угловой скорости, рад/с	2,01335
Отклонение конечной зарегистрированной точки от начальной, м	0,17445

Показатели таблицы 7 рассчитаны по журналу экспериментальной проверки `square_hil.csv`. Значения `seq_ok` и `status` подтверждают корректность обмена между Webots и STM32 в течение зарегистрированного эксперимента: ответные кадры согласованы с переданными входными данными, а ошибок обработки прошивкой не зафиксировано.

Основным результатом проверки является то, что воздействия, применяемые к модели Webots, рассчитывались на STM32 и возвращались через выходные кадры. Следовательно, эксперимент подтверждает выполнение разработанной программы управления на микроконтроллерной платформе в составе замкнутого контура с моделируемым объектом управления.

Отклонение конечной точки характеризует точность выполнения выбранного сценария, но не является критерием самого факта работоспособности встраиваемой реализации. Цель данного этапа — подтвердить выполнение программы на STM32 и её взаимодействие с моделью объекта управления — достигнута.

4.5. Границы применимости результатов

Проведённая проверка относится к встраиваемой программе управления и её взаимодействию с моделируемым объектом. Физически в эксперименте используются STM32 и канал связи с компьютером; мобильная платформа, датчики и исполнительные механизмы представлены моделью Webots.

Поэтому полученные результаты подтверждают выполнение программы управления на STM32, корректность обмена и работоспособность замкнутого контура с моделью объекта. Они не характеризуют реальные двигатели, механическую конструкцию, физические датчики, питание и взаимодействие колёс с поверхностью, поскольку испытание данных элементов в объём настоящей работы не входит.

Таким образом, выполненная аппаратно-программная проверка является достаточной для подтверждения результата в пределах поставленной задачи: разработанная программа управления выполняется на STM32 и обеспечивает управление моделью четырёхколёсного мобильного робота в замкнутом контуре.

4.6. Выводы

В данной главе рассмотрена встраиваемая реализация программы управления и её аппаратно-программная проверка на платформе STM32.

В реализованном контуре среда Webots воспроизводит модель мобильной платформы и предоставляет данные о её состоянии, а STM32 выполняет расчёт управляющих воздействий. Для обмена используются кадры SensorFrame и ControlFrame, предусматривающие сопоставление шага обработки и контроль целостности данных. В составе прошивки выполняются основные компоненты разработанной программы управления: Robot, FFModel, регуляторы и RobotController.

В ходе проверки зарегистрировано 1653 шага обмена. Все ответные кадры соответствовали переданным входным кадрам, кадров с состоянием ошибки не зарегистрировано. Среднеквадратическая ошибка линейной скорости составила 0,01884 м/с, среднеквадратическая ошибка угловой скорости — 0,22209 рад/с.

Полученные результаты подтверждают, что разработанная программа управления включена в состав прошивки STM32, выполняется на микроконтроллере и обеспечивает управление моделируемой мобильной платформой в принятом аппаратно-программном контуре. Испытание физической ходовой части робота находится за пределами настоящей работы.

Заключение

В ходе выполнения выпускной квалификационной работы разработана программа управления ходовой частью четырёхколёсного мобильного робота с дифференциальной схемой привода, предназначенная для выполнения на микроконтроллерной платформе STM32.

В работе рассмотрена ходовая часть мобильной платформы как объект управления, построена предварительная математическая модель, описывающая общие кинематические свойства платформы с дифференциальным приводом, сформулирована задача скоростного управления и разработан закон управления по заданным линейной и угловой скоростям. На этой основе реализована программная система, обеспечивающая получение данных о скоростном состоянии платформы, вычисление управляющих воздействий и их передачу исполнительному уровню.

Архитектура программы построена с разделением вычислительной логики, средств аппаратной абстракции и платформенно-зависимых компонентов. Такое построение позволило использовать один и тот же вычислительный контур при испытаниях в среде Webots и в составе прошивки микроконтроллера STM32, изменяя только реализацию взаимодействия с измерительной и исполнительной частями.

Корректность программных компонентов подтверждена модульным тестированием. Работоспособность их совместного функционирования в составе замкнутого контура управления проверена в среде моделирования Webots при выполнении прямолинейного движения, поворота на месте и движения по квадратной траектории. Полученные результаты показали устойчивое воспроизведение заданных скоростных режимов и отсутствие нарастающих отклонений. При этом наличие ошибки конечного положения на замкнутом маршруте соответствует границам разработанной системы, не включающей позиционный контур управления.

Дополнительно выполнена аппаратно-программная проверка, в которой модель мобильной платформы воспроизводилась средой Webots, а вычисление управляющих воздействий выполнялось на физическом микроконтроллере STM32

Таким образом, цель работы достигнута: разработана и экспериментально проверена программа управления ходовой частью четырёхколёсного мобильного робота на микроконтроллерной платформе STM32.

Список источников

1. Бесекерский В. А., Попов Е. П. Теория систем автоматического управления. — 4-е изд., перераб. и доп. — СПб. : Профессия, 2007. — 747 с.
2. Давыдов О. И., Платонов А. К. Алгоритм управления дифференциальным приводом мобильного робота РБ-2 // Препринты ИПМ им. М. В. Келдыша. — 2015. — № 25. — 16 с.
3. Денисенко В. В. ПИД-регуляторы: принципы построения и модификации // Современные технологии автоматизации. — 2006. — № 4. — С. 66–74.
4. Иванов В. А., Ющенко А. С. Теория дискретных систем автоматического управления : учебное пособие для вузов / под ред. Е. П. Попова. — М. : Наука, 1983. — 335 с.
5. Кампион Г., Бастен Ж., Д'Андреа-Новель Б. Структурные свойства и классификация кинематических и динамических моделей колесных мобильных роботов // Нелинейная динамика. — 2011. — Т. 7, № 4. — С. 733–769.
6. Ключарёв А. А., Кочин К. А., Фоменкова А. А. Программирование микроконтроллеров STM32 : учебное пособие. — СПб. : ГУАП, 2023. — 196 с.
7. Липаев В. В. Программная инженерия. Методологические основы : учебник для вузов. — М. : ТЕИС, 2006. — 605 с.
8. Мартыненко Ю. Г. Управление движением мобильных колёсных роботов // Фундаментальная и прикладная математика. — 2005. — Т. 11, вып. 8. — С. 29–80.

9. Орлов С. А., Цилькер Б. Я. Технологии разработки программного обеспечения : учебник для вузов. — 4-е изд. — СПб. : Питер, 2012. — 608 с.
10. Усольцев А. А. Электрический привод : учебное пособие. — СПб. : НИУ ИТМО, 2012. — 238 с.
11. Юревич Е. И. Основы робототехники : учебник для студентов вузов. — Л. : Машиностроение, 1985. — 270 с.
12. Beningo J. Reusable Firmware Development: A Practical Approach to APIs, HALs and Drivers. — Berkeley, CA : Apress, 2017. — XXII, 308 p. — DOI: 10.1007/978-1-4842-3297-2.
13. Chen X., Salem M., Das T., Chen X. Real Time Software-in-the-Loop Simulation for Control Performance Validation // SIMULATION. — 2008. — Vol. 84, No. 8–9. — P. 457–471. — DOI: 10.1177/0037549708097420.
14. CMake Documentation. cmake-toolchains(7): CMake Toolchains Reference [Электронный ресурс]. — <https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html> (дата обращения: 23.06.2026).
15. Cyberbotics. Webots Documentation. Controller Programming [Электронный ресурс]. — URL: <https://cyberbotics.com/doc/guide/controller-programming> (дата обращения: 23.06.2026).
16. Cyberbotics. Webots Reference Manual. Release R2025a [Электронный ресурс]. — <https://cyberbotics.com/doc/reference/index> (дата обращения: 23.06.2026).
17. GoogleTest. GoogleTest User's Guide [Электронный ресурс]. — <https://google.github.io/googletest/> (дата обращения: 23.06.2026).

18. Isermann R., Schaffnit J., Sinsel S. Hardware-in-the-loop simulation for the design and testing of engine-control systems // Control Engineering Practice. — 1999. — Vol. 7, No. 5. — P. 643–653. — DOI: 10.1016/S0967-0661(98)00205-6.
19. STMicroelectronics. DS10314. STM32F411xC, STM32F411xE: Datasheet [Электронный ресурс]. — <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf> (дата обращения: 23.06.2026).
20. STMicroelectronics. RM0383. STM32F411xC/E advanced Arm-based 32-bit MCUs: Reference manual [Электронный ресурс]. — https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xc-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (дата обращения: 23.06.2026).
21. STMicroelectronics. UM1725. Description of STM32F4 HAL and low-layer drivers: User manual [Электронный ресурс]. — https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-lowlayer-drivers-stmicroelectronics.pdf (дата обращения: 23.06.2026).